

Process Scheduling — Chapter 5



叶冠华

g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)



北京邮电大学

■ 进程

- ❑ 进程状态 (PS)：就绪、运行、阻塞(也称等待)、(创建、终止)
- ❑ 进程状态的转换：
 - 运行→(阻塞→)就绪→运行、(创建→就绪、运行→终止)
 - 就绪/阻塞区别：获得CPU后是否能立即运行(就绪√ 阻塞×)
- ❑ 进程的组成：进程控制块(PCB)(位于内核)、代码段、数据段
- ❑ PCB：进程描述信息、进程控制信息、CPU上下文、分配的资源等
- ❑ 进程的创建：申请空白PCB → 分配资源 → 初始化PCB → 创建态
- ❑ 进程上下文切换：
 - 将待挂起进程CPU上下文保存到PCB(将该PCB移入就绪/阻塞队列)
 - 将选择另一个进程执行，用其PCB更新CPU上下文
 - CPU跳转到新进程PC的位置开始执行
- ❑ 进程通信：共享存储、消息传递、管道通信、信号

■ 线程

- ▣ 轻量级进程，不拥有系统资源，有线程控制块TCB（执行、就绪、阻塞态）
- ▣ 一个进程中的多个线程之间也能并发执行
- ▣ 同一进程中的不同线程共享进程的地址空间和资源
- ▣ 多线程进程，可将进程中的多个线程分配到多个CPU上执行。
- ▣ 线程的实现：
 - 用户级线程(ULT)
 - 核级线程(KLT)
 - ULT-KLT组合使用：多对一、一对一、多对多

CHAPTER OBJECTIVES

- To introduce CPU-scheduling, which is the basis for multiprogrammed operating systems.
- To describe various CPU-scheduling algorithms.
- To discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system.
- To examine the scheduling algorithms of several operating systems.

Chapter 5 Process Scheduling

I. 5.1 Basic Concept (resource allocation: why, who, when)

5.2 Scheduling criteria:
CPU utilization, throughput;
turnaround time,
waiting time,
respond time;
(fairness, deadline, ...)

II. How to evaluate scheduling algorithms/methods (good or bad ?)

5.8 Algorithm evaluation

Appendix 5-3 Scheduling in Linux
(priorities, O(1), CFS)

VII. Operating-system examples

5.7 Unix, Windows

VIII. 附录:
Appendix 5-1 基于指数平均的CPU burst预测;
Appendix 5-2 Priority-based scheduling in Unix

IX. Examples and exercises:
操作系统概念 (英文) ——5-Process Scheduling-例题与作业

多核多线程编程及性能对比分析

X. Linux Experiments

First-Come First-Served(FCFS)

Shortest-Job-First(SJF), SRTF

Priority
Scheduling

preemptive, nonpreemptive

Round Robin(RR)

Multiple Queue, Multiple Feedback Queue

Highest Response Rate First(HRRF)

III. *5.3 Scheduling algorithms*

IV. 5.4 Thread scheduling

Two-level scheduling for user threads

V. 5.5 Multiple-processor scheduling (two-level scheduling)

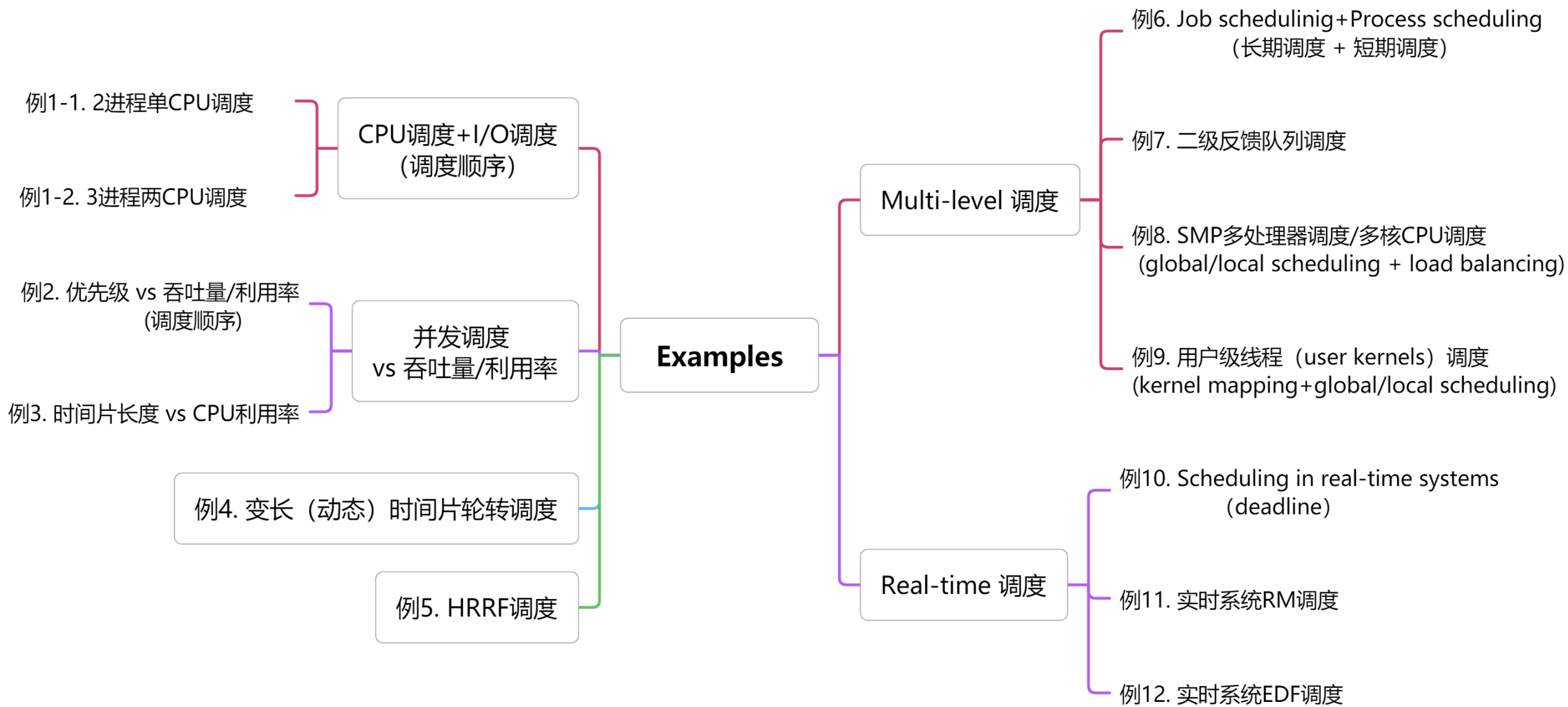
homogeneous, asymmetric,
processor affinity, load balancing

scheduling for multicore processor

VI. 5.6 Real-time CPU scheduling

priority-based and preemptive scheduling

rate monotonic scheduling,
earliest deadline first scheduling(EDF)
proportional share scheduling



Terminology

monopolize, multiplex	独占, 复用
CPU-burst, I/O burst	CPU执行/突发, I/O执行/突发
CPU-bound, computation-intensive	CPU-受限, 计算密集型
I/O-bound, I/O-intensive	I/O-受限, I/O密集型
dispatcher	分配器,调度器
preemptive	抢占式
dispatch latency	调度延迟
Scheduling Criteria	调度准则
CPU utilization, throughput, turnaround time, waiting time, response time	CPU占用率/使用率, 吞吐量, 周转时间, 等待时间, 响应时间

fairness, usage, real-time, power-consumption	公平性,资源利用, 实时性, 能耗
Shortest-Job-First (SJR), Shortest-Remaining-Time- First(SRTF), Priority scheduling, Round Robin(RR), time quantum, time slice, Multilevel feedback queue, Highest Response Rate First (HRRF)	最短作业优先, 最短剩余时间优先, 优先级调度, 时间片轮转, 时间量,时间片 多级反馈队列, 最高响应比优先

Terminology

Process-contention scope (PCS), System-contention scope (SCS)	进程竞争范围, 系统竞争范围
Homogeneous processor Asymmetric multiprocessing Symmetric multiprocessing(SMP)	异构处理器, 非对称多处理, 对称多处理,
processor affinity, load balancing	处理器亲和, 负载均衡
Hyperthread, Multicore Processor	超线程, 多核处理器
memory stall	内存停顿
time-critical system, deadline	时间紧迫系统, 截止期
Rate Monotonic Scheduling, Earliest Deadline First Scheduling(EDF) Proportional Share Scheduling	速率单调调度, 最早截止期优先调度, 比例分享调度

5.1 Basic Concepts

- Why scheduling needed
 - ❑ multiple processes concurrently run in systems, a process is not allowed to monopolize/独占 the CPU and other resources such as I/O devices
 - ❑ processes have to multiplex/复用 one or more CPU (CPU cores), and occupy CPU for execution in order
 - also for other resources
 - ❑ thus process scheduling, i.e. CPU scheduling and IO scheduling are needed
- CPU scheduling
 - ❑ OS **selects** running entities in main memory (i.e., processes or threads in the ready queue) according to some criteria, **allocates** CPU to the selected running entities, and then **enables** them to run on CPU
 - ❑ selecting + allocating + enabling, in kernel mode
 - ❑ known as short-term scheduling

Basic Concepts

- Process execution cycle
 - ▣ process execution consists of a cycle of CPU execution and I/O wait, i.e., CPU burst/执行 and I/O burst/执行
- When a CPU burst occurs, the process is in the **running** state;
- When a I/O burst occurs, the process is in the **waiting** state
- CPU-burst times distribution, Fig.5.2

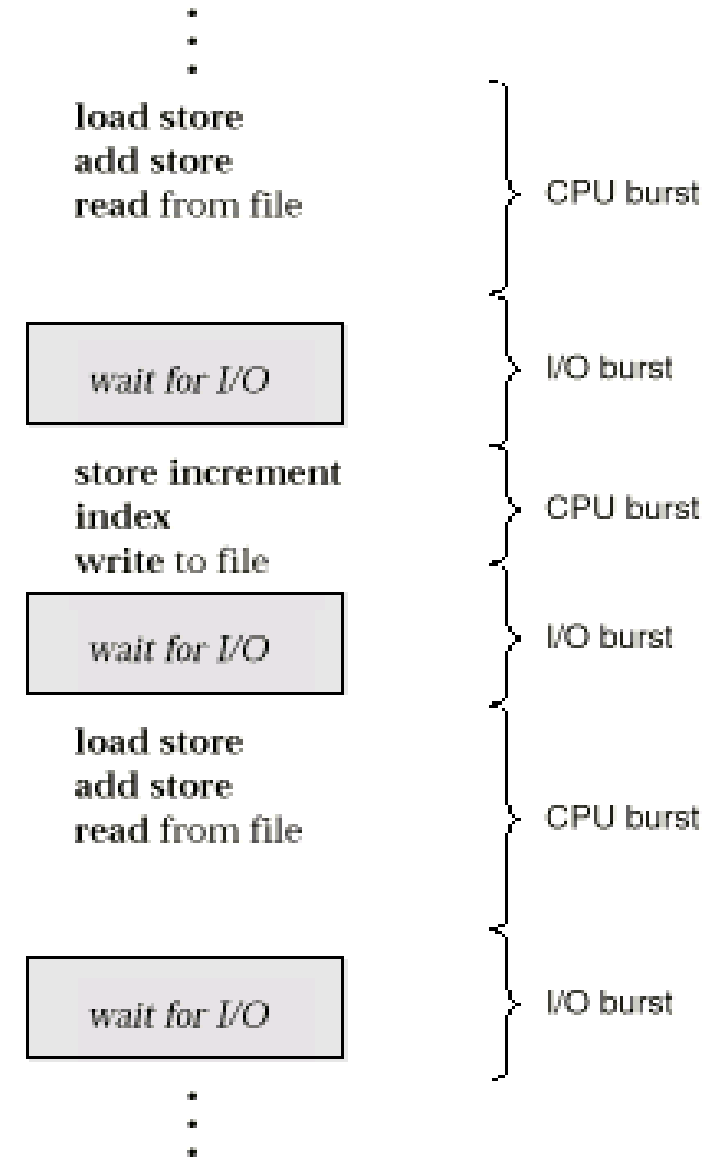


Fig. 5.1 Alternating Sequence of CPU And I/O Bursts

Basic Concepts

- CPU-bound process might have a few very long CPU bursts
 - ▣ computation-intensive, 计算密集型
- When a I/O-bound process might have many very short CPU bursts and some long I/O bursts
 - ▣ I/O密集型, I/O intensive, e.g. DB access

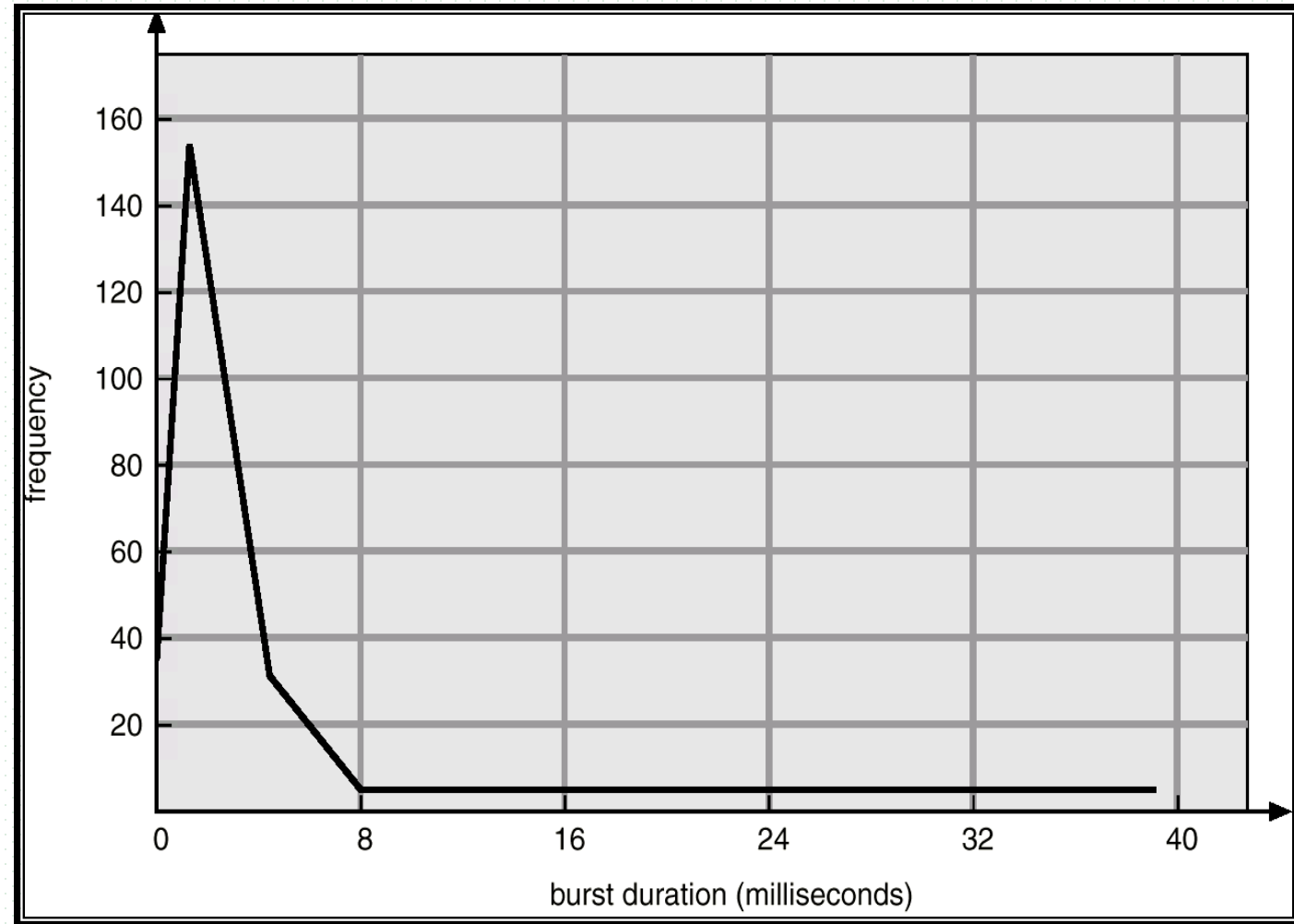


Fig. 5.2 CPU burst distribution
— histogram of CPU- burst Times

Who conduct CPU Scheduling

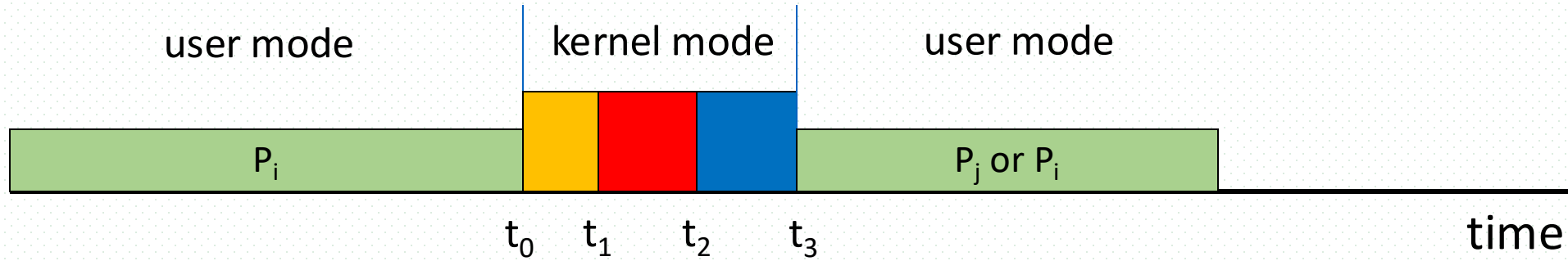
- As components of OS, scheduler (§5.1.2) and dispatcher (§ 5.1.4, 调度器、分配器) are responsible for CPU scheduling
 - ▣ Linux: kernel/sched.c
- The (short-term) **scheduler** is responsible for CPU scheduling
 - ▣ selects from the processes in memory that are ready to execute
 - using **scheduling algorithms**
 - ▣ allocates the CPU to one of them
 - *组织和维护(就绪)进程/线程队列
- The **dispatcher** gives control of the CPU to the process selected by the scheduler, i.e. starts the selected process
 - ▣ process context switching
 - ▣ switching to user mode
 - ▣ jumping to the proper location in the user program to restart that program

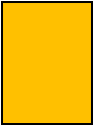
When scheduling occurs

- When CPU is idle or some events occurs in systems (e.g. real-time system), the OS scheduler should take the control of CPU , and then make scheduling decisions
 - process terminates
 - the running process switches from running to waiting state (for example, I/O requests)
 - the running process switches from running to ready state (e.g., interrupts occurs, or the timeslot is out)
 - (maybe) switches from waiting to ready (e.g. , completion of I/O)
 - Scheduling under condition 1 and 2 is nonpreemptive (非抢占式), because the CPU becomes in idle
 - All other scheduling is preemptive (抢占式) , often used in real-time systems
- 

CPU mode/process context switching in scheduling

- CPU scheduling results in CPU switching between user mode and kernel mode



 : system calls, interrupt handling ,etc., resulting in process context saving
, under four circumstances (as described in the textbook)

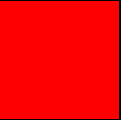
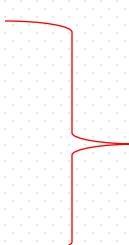
 : scheduler,  : dispatcher

Fig. 5.0.1 Illustration of **mode switching** in CPU scheduling

5.2 Scheduling Criteria/准则

- Scheduler aims to optimize criteria

- maximize CPU utilization/使用率/利用率
 - maximize throughput/吞吐量
- 
- system level criteria, specific to the whole processes in system

- minimize turnaround time/周转时间
 - minimize waiting time/等待时间
 - minimize response time/响应时间
- 
- process level criteria, specific to a particular process

- Others

- 公平性fairness, 资源利用usage, 实时性real-time, 能耗power-consumption

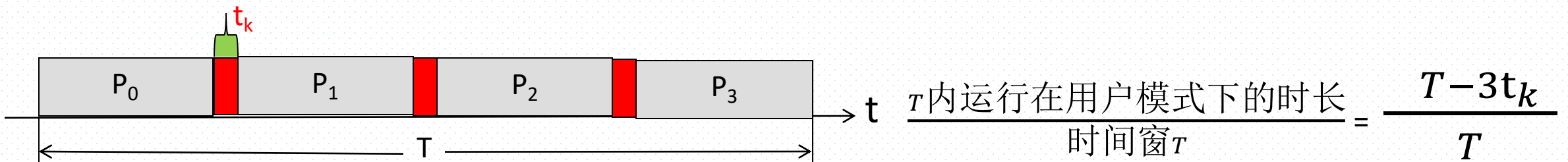
CPU utilization

■ CPU utilization

- keep the CPU as busy as possible
- keeping the CPU in user mode as possible, refer to Fig. 5.0.1
 - CPU运行在用户模式下的时间占比

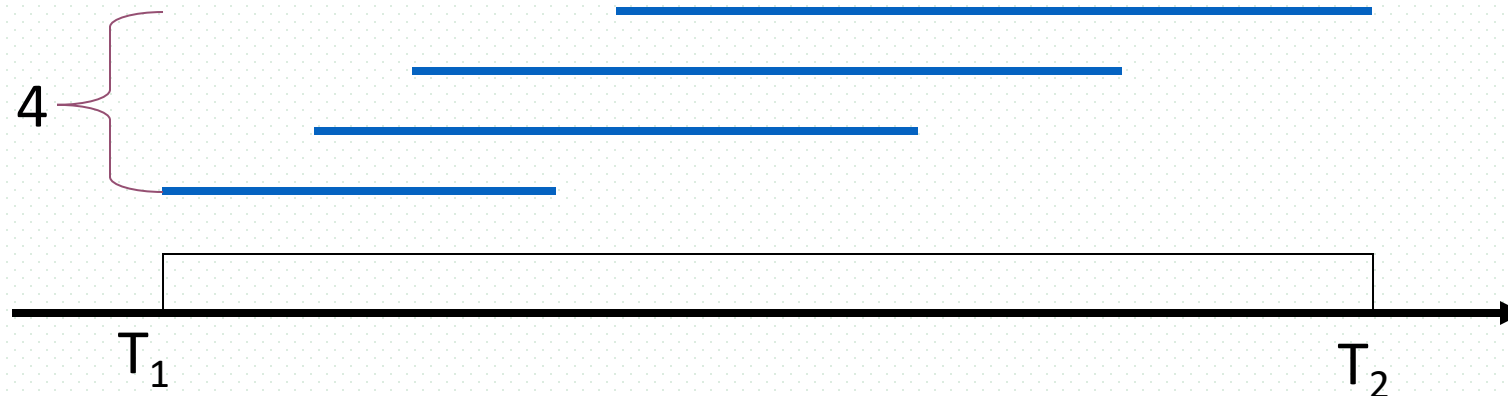
$$\frac{\text{amount of time during which user processes run on CPU}}{\text{amount of time during which user and OS processes run on CPU}} \times 100\%$$

用户应用程序，非OS的系统程序（e.g. 编译程序）



Throughput

- Throughput/吞吐量
 - ▣ the number of processes that complete their execution **per time unit**
- E.g.
throughput
= number of processes terminated in $[T_1, T_2]$, e.g. $= 4 / (T_2 - T_1)$



Turnaround Time, Waiting Time

- Turnaround time/周转时间
 - ▣ amount of time taken to execute/complete a particular process
 - ▣ i.e. the length of the process lifetime cycle, the time the process resides in ready, running, waiting, andstates
- Waiting time
 - ▣ the sum of the periods spent waiting in the ready queue
 - ▣ i.e. amount of time a process has been waiting in the ready_queue

Turnaround Time, Waiting Time

- **turnaround time:** $[t_0, t_8]$
- **waiting time:** $[t_0, t_1] + [t_3, t_4] + [t_6, t_7]$

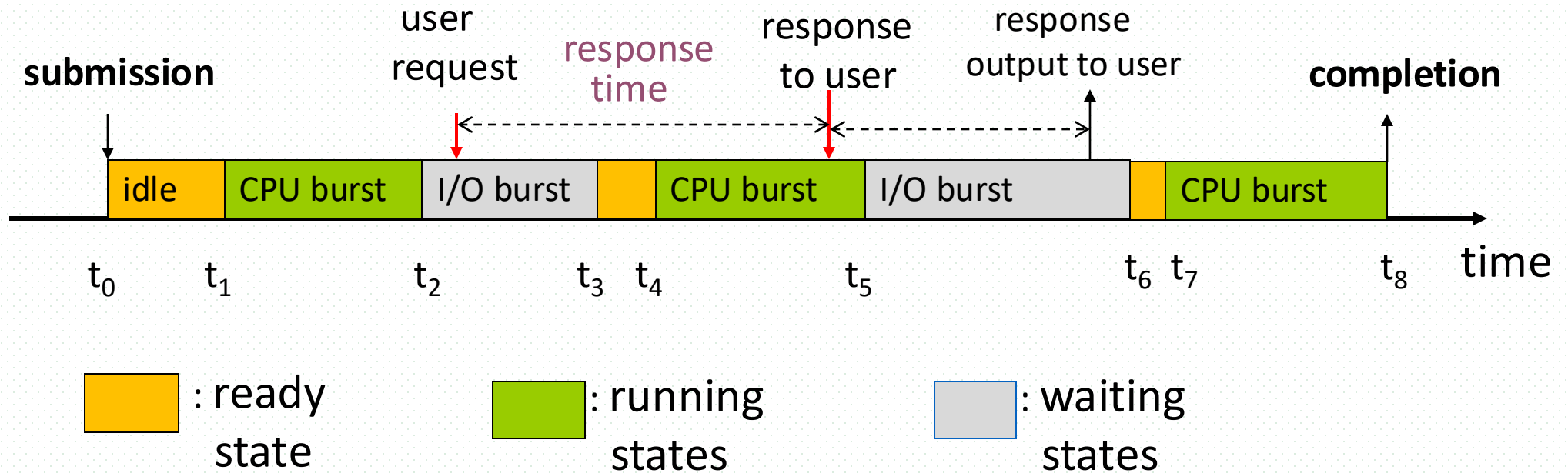
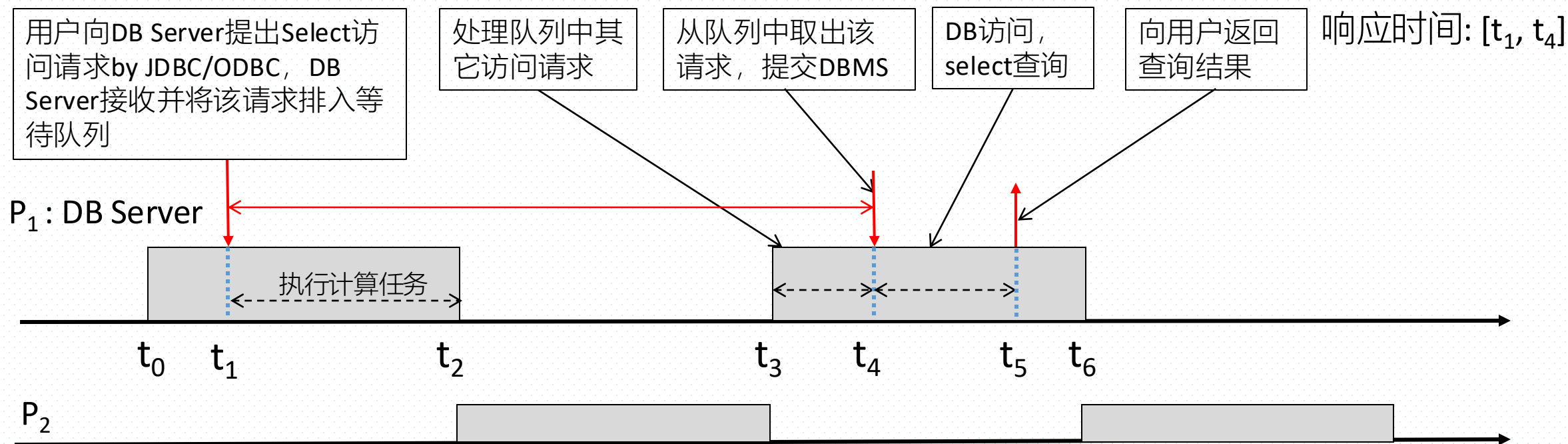


Fig.5.0.2 Lifetime cycle of a process

Response Time

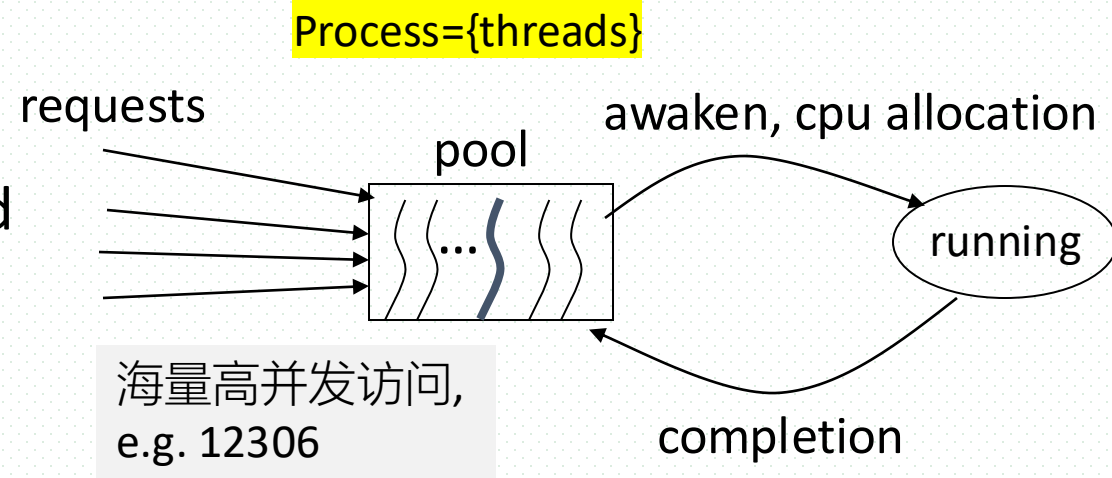
■ Response time/响应时间

- ❑ in interactive systems, the time from the submission of a request until the first response is produced
- ❑ the time it takes to **start responding**, not the time it takes to output the response
- ❑ E.g. 1 query request in DBS



Response Time

- E.g. 2 A service request arrives at the thread pool
 - ▣ the response time is the interval from the time when the request arrives to that at which a thread is selected to serve the request



Response Time: threads in pool

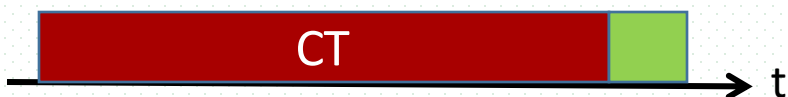
当 $L > M$ 时, e.g. $L=5, M=4$, 部分用户请求的响应时间较大

- 根据服务器端的CPU核的数目 N 、线程需要执行的任务/进程类型, 确定线程数目 M

- 任务类型

- CPU密集型, CPU-intensive, CPU-bound

- e.g. 科学计算, 矩阵计算

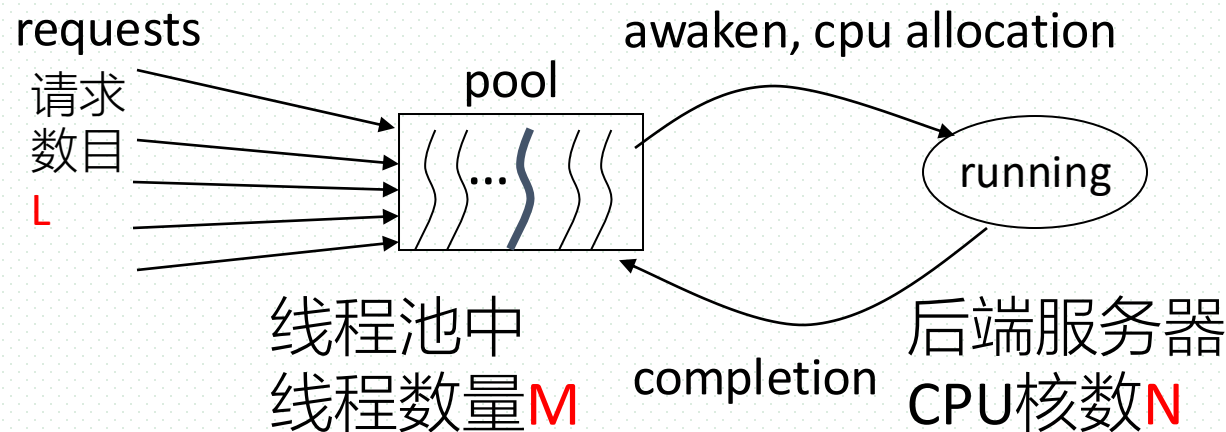


- I/O密集型, I/O-intensive, I/O-bound

- e.g. 数据库访问磁盘I/O, 网络I/O



海量高并发访问



M设置方式

- CPU密集型任务:

$$M = N$$

- I/O密集型任务:

$$M = N * (1 + \text{WT} / \text{CT})$$
$$= \text{e.g. } 4 * (1 + 75 / 25)$$

CT: computing time
WT: waiting/I/O time

CT: 计算任务执行时间
WT: I/O操作执行时间

Example-Question

- In a multiprogramming batch system, there are two processes P_1 and P_2 concurrently running on a single CPU. P_2 enters into the system 30ms later than P_1 , and their executing traces, i.e. alternating sequences of CPU bursts and I/O bursts, are as follows
 - ▣ P_1 : computing, 50ms $\rightarrow$ I/O operation, 100ms $\rightarrow$ computing, 100ms
 $\rightarrow$ I/O operation, 80ms
 - ▣ P_2 : computing, 150ms $\rightarrow$ I/O operation, 50ms $\rightarrow$ computing, 60ms
- It is assumed that time costs of CPU scheduling and process switch are omitted, illustrate the executing traces of P_1 and P_2 by charts, and compute
 - ▣ 1) the maximal throughput for completing these two processes
 - ▣ 2) the waiting times for P_1 and P_2 respectively

5.3 Scheduling Algorithms

- First-Come First-Served (FCFS)
- Shortest-Job-First (SJR)
 - non-preemptive SJF
 - preemptive SJF
 - Shortest-Remaining-Time-First, **SRTF**
- Priority scheduling
 - preemptive, nonpreemptive
- Round Robin (RR, 时间片轮转调度)
- Multilevel queue
- Multilevel feedback queue
- Highest Response Rate First (HRRF)/最高响应比优先
- Scheduling in real-time system

First-Come First-Served (FCFS)

- Principle

- the process that requests the CPU first is allocated the CPU first

- Implementation

- the new created process is inserted into a **FIFO ready queue**, i.e. inserted into the tail of the ready queue
- scheduler selects **the first** process in the ready queue

Example

- There are three processes P_1 , P_2 , P_3 , the CPU bursts of each process are

Process (CPU and I/O) Burst Time

P_1 24

P_2 3

P_3 3

- suppose that the processes arrive in the order: P_1 , P_2 , P_3

The **Gantt Chart** for the schedule is:



- waiting time for $P_1 = 0$, $P_2 = 24$, $P_3 = 27$
- average waiting time: $(0 + 24 + 27)/3 = 17$

Example

- suppose that the processes arrive in the order

P_2, P_3, P_1 .

- the Gantt chart for the schedule is:



- waiting time for $P_1 = 6$, $P_2 = 0$, $P_3 = 3$
- average waiting time: $(6 + 0 + 3)/3 = 3$
 - much better than previous case, because the processes with shorter bursts run at first

First-Come First-Served (FCFS)

- FCFS Features

- ▣ FCFS is nonpreemptive
- ▣ the average waiting time under FCFS is generally not minimal, and may vary substantially if the process CPU-burst times vary greatly
- ▣ FCFS **prefers to** the processes with shorter CPU-burst
 - 优先调度the processes with shorter CPU-burst将获得较好的性能, 因为其它进程等待时间短

Shortest-Job-First (SJF) scheduling

■ Principles

- associate with each process the length of its **next CPU burst**
- use these lengths to schedule the process with the shortest time, i.e., allocate CPU to the process with the **smallest** next CPU burst
- if two processes have the same next CPU bursts, FCFS scheduling is used

■ Implementation

- nonpreemptive
 - once CPU given to the process it cannot be preempted until completes its CPU burst
- preemptive
 - if a new process (as an **external event**) arrives with CPU burst length less than remaining time of current executing process, preempting and scheduling occurs, according to the remaining burst of the preempted process

当参与调度的2个进程的next CPU burst、剩余时间相同时，按照FCFS策略，选择先创建的进程

Shortest-Job-First (SJF) scheduling

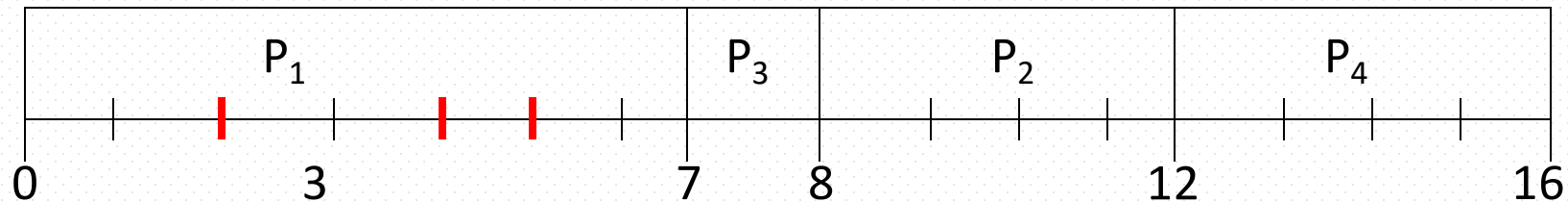
- Preemptive SJF is known as the Shortest-Remaining-Time-First (SRTF)
- Feature
 - with respect to **minimum average waiting time** for a given set of processes, SJF is optimal

Example

- E.g. SJF (non-preemptive)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P ₁	0.0	7
P ₂	2.0	4
P ₃	4.0	1
P ₄	5.0	4

参与调度的进程P₂、P₄的next CPU burst相同，按照FCFS策略，选择先创建的P₂

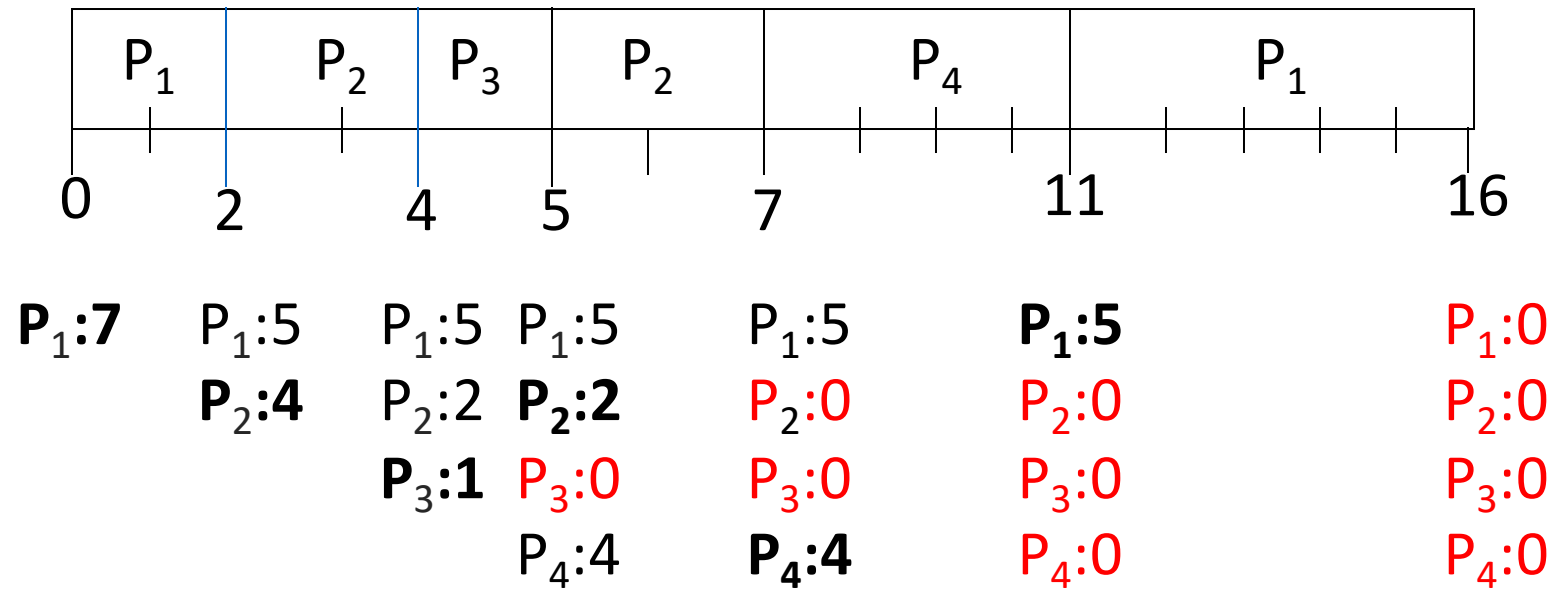


- average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

- E.g. preemptive SJF, SRTF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
----------------	---------------------	-------------------

P ₁	0.0	7
P ₂	2.0	4
P ₃	4.0	1
P ₄	5.0	4



□ average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

Shortest-Job-First (SJF) scheduling

- How to determining the length of next CPU burst
 - ▣ can only estimate the length
 - ▣ can be predicted by using the length of previous CPU bursts, using exponential averaging, using the formula given in the textbook
 - ▣ for more details about exponential averaging prediction (指数平均/平滑预测)
 - refer to Appendix 5-1 基于指数平均的CPU burst预测 ▶ 【略】

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$$

1. t_n = actual length of n^{th} CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. $\alpha, 0 \leq \alpha \leq 1$

Priority scheduling

■ Principles

- a priority number (integer) is associated with each process the CPU is allocated to the process with the highest priority
- often, smallest integer $\equiv$ highest priority

■ Implementation

- nonpreemptive
 - insert the new process into the ready queue, and wait for CPU scheduling on the basis of priorities
- preemptive
 - preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process
 - e.g. Thread scheduling in Windows, refer to 5.7 Scheduling in Windows

Example

<u>Process</u>	<u>Burst Time</u>	<u>Priority Number</u>
P ₁	10	3
P ₂	1	1
P ₃	2	4
P ₄	1	5
P ₅	5	2

Smaller priority **numbers** imply higher scheduling priorities

5个进程同时进入系统



Priority Scheduling

- Types of priority
 - ▣ static priority
 - the priority remains unchanged during process lifetime
 - ▣ dynamic priority
 - the priority may vary, e.g. in Linux
- How to determine the priority
 - ▣ defined internally
 - set by OS, according to process properties
 - ▣ defined externally
 - can be set and changed by users 

Priority Scheduling

- FCFS and SJF can be viewed as priority-based scheduling 
 - ▣ in FCFS, the priority is the waiting time of the process from its being created
 - ▣ in SJF, the priority is the predicted next CPU burst time

Priority in Linux

- Linux task_struct 中，表示进程/task优先级的字段
 - ▣ static_prio
 - 静态优先级，可以调用 nice 系统调用，直接修改
 - 取值范围 100~139
 - ▣ rt_priority
 - 实时优先级，取值范围 0~99
 - ▣ prio
 - 动态优先级
 - ▣ normal_prio
 - 其值取决于静态优先级和调度策略

Priority Scheduling

- Features
 - ▣ commonly used in modern operating systems
 - ▣ may be indefinite blocking/starvation
- Starvation/饿死 Problem
 - ▣ low priority processes may never execute
 - ▣ solution: aging
 - as time progresses, increase the priority of the process, i.e. dynamic priority is adopted
- E.g 1. Dynamic priority-based scheduling in Unix
 - ▣ refer to Appendix B Priority-based scheduling in Unix 
- E.g 2. CPU scheduling in Linux
 - ▣ refer to Linux as Case Studies-6-Process scheduling
 - O(1), CFS

Round Robin- RR/(时间片)轮转调度

■ Principles

- ❑ CPU time is divided into a series of small time units, called **time quantum/时间量** or **time slice/时间片**
 - fixed-length slice
 - non-fixed-length, dynamic slice, e.g. CFS scheduling in Linux
- ❑ each process in ready queues is allocated a time slice, usually 10-100 milliseconds, and begins to run on CPU
- ❑ scheduler selects **the first process (head process)** in the ready queue, this process is allocated a CPU time slice
- ❑ after this time slice has elapsed, the process is preempted and added to **the tail** of the ready queue, and the CPU is allocated to the next process in the ready queue

Round Robin- RR, 时间片轮转

■ Features

- typically, higher average **waiting time** than SJF, but better **response time that guarantees real-time property**
 - 可以保证进程实时性
- commonly used in **time-sharing** systems

■ 目标

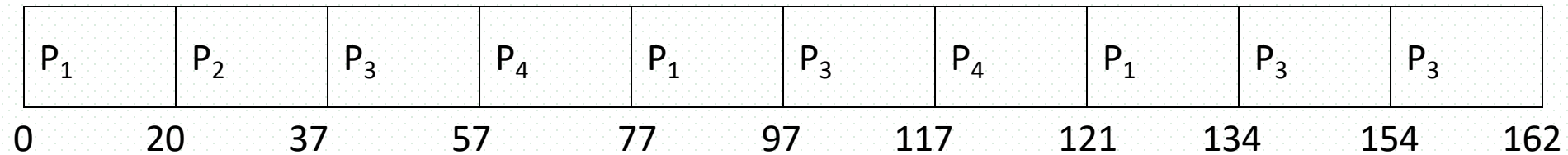
- 进程等概率获得被调度机会, 以维持公平性 (fairness)和实时性(real-time)
- 进程优先级越高, 获得的时间片越长, 享有更多CPU使用权

Example

- E.g. RR with Time Quantum = 20

<u>Process</u>	<u>Burst Time</u>
P ₁	53
P ₂	17
P ₃	68
P ₄	24

- The Gantt chart is



Time Quantum vs Performance: context switch time

- q large $\Rightarrow$ FCFS
- q small $\Rightarrow$ the number of context switch increase
 - so q must be large with respect to context switch, otherwise overhead is too high

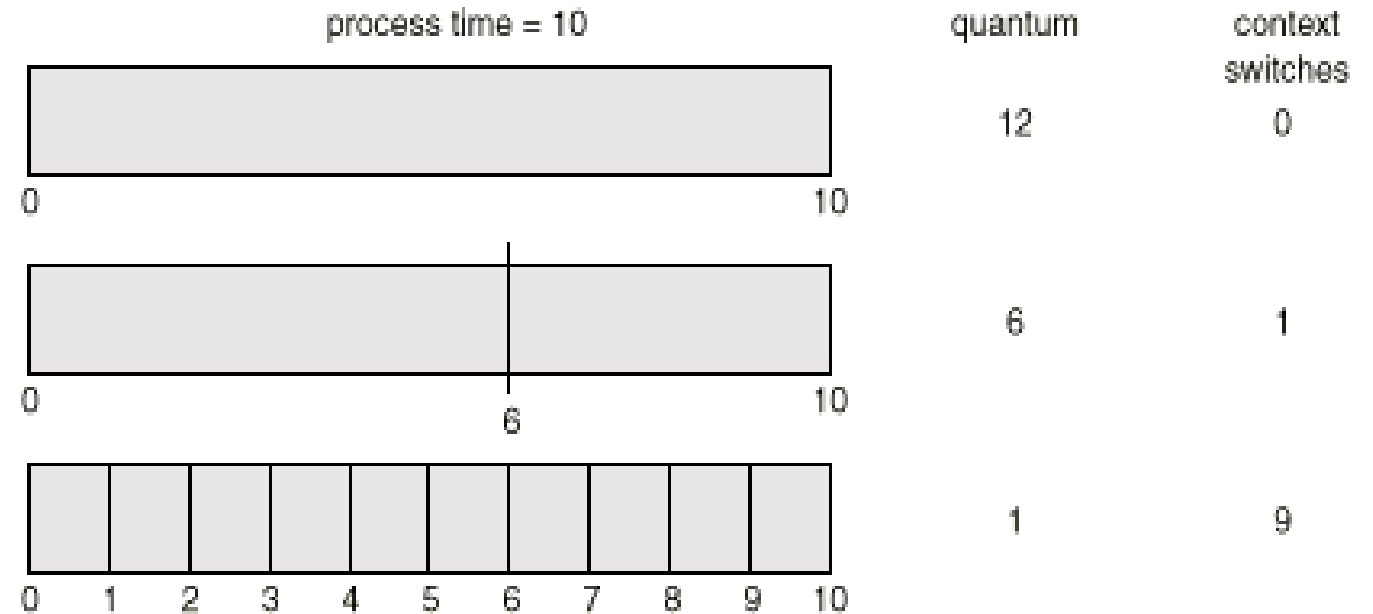


Fig. 5.4 Time Quantum and Context Switch Time

Time Quantum vs Performance: turnaround time

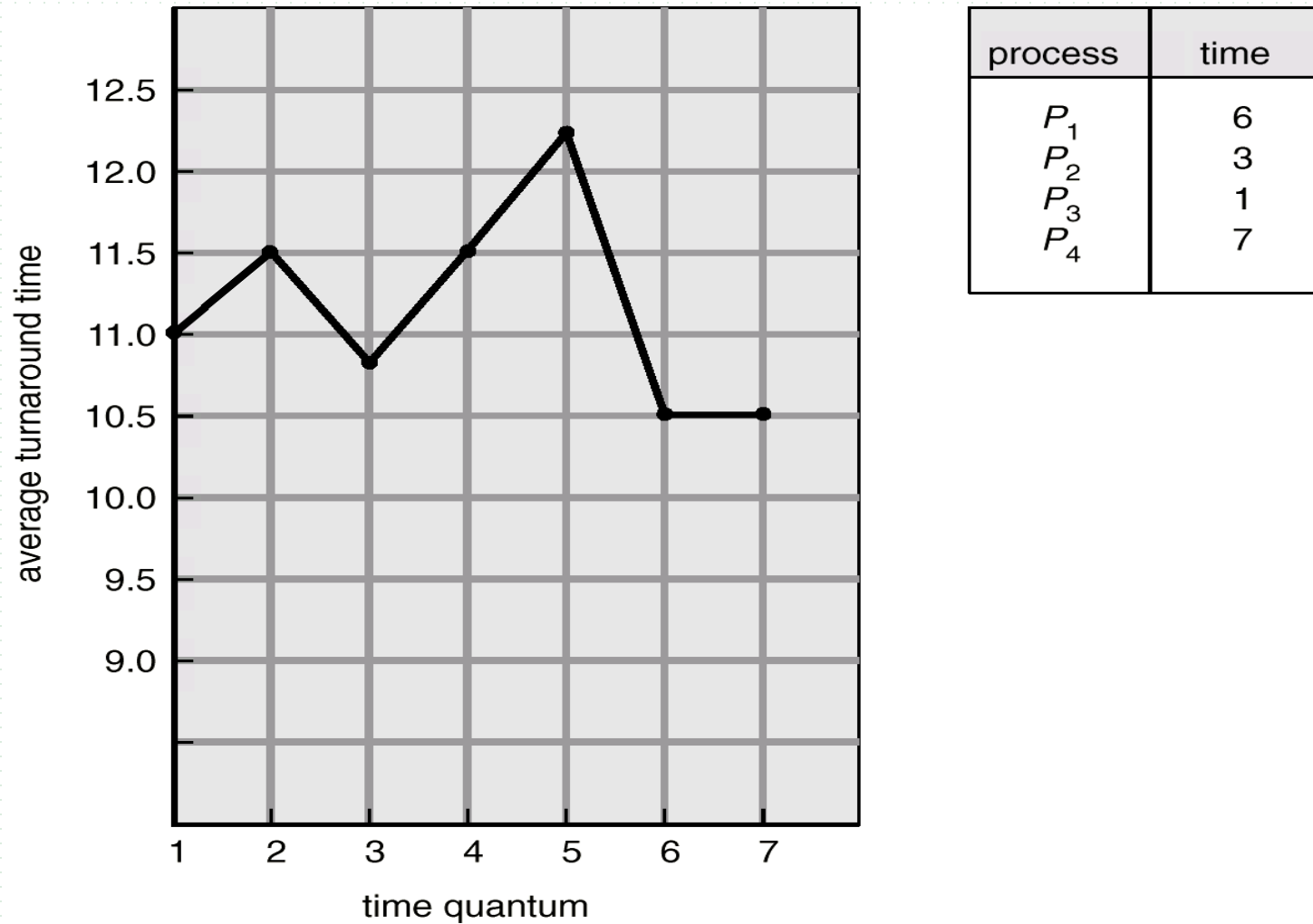


Fig. 5.5 Turnaround Time Varies With The Time Quantum

Time quantum vs Performance

- If there are n processes in the ready queue and the time quantum is q , then
 - ▣ each process gets $1/n$ of the CPU time in chunks of **at most q time units** at once
 - ▣ no process waits more than $(n-1)q$ time units
- 问题：
 - n 个进程，要求每个进程的**响应时间**（？）至多为 m ，则时间片 q 至多应是多少？
 - ▣ 未必能保证，**必要但非充分条件**
 - ▣ 响应时间不仅仅取决于时间片，还取决于进程内部业务逻辑

Variable-length/Dynamic Time Slice

- CPU time consists of fixed-length scheduling periods; in each period, CPU time is divided into a series of variable-length time slices, with their lengths proportional to process priorities, and each slice is allocated to a process
 - 根据各个进程的优先级，确定进程获得的时间片长度，优先级越高，获得的时间片长度越长

- E.g.1 period=100

process	P_0	P_1	P_2
priority	1	2	3
slice length	$100 * (1/6)$	$100 * (2/6)$	$100 * (3/6)$

- Why variable-length slice ?

兼顾实时性和公平性(fairness)

- RR保证进程等概率获得被调度机会，以维持实时性
- 公平性：在保证实时性前提下，优先级高的进程应获得较多CPU资源

Variable-length/Dynamic Time Slice

- E.g.2 Time slice/quantum in Solaris ▶
- E.g.3 CFS (Complete Fairness Scheduling, 完全公平调度) in Linux
 - ▣ refer to Appendix 5-3 Scheduling in Linux ▶

$$time_slice_i = \underbrace{sched_period}_{\text{调度周期}} * \frac{weight_i}{weight_rq}$$



- E.g.4 Example 4 变长/动态时间片轮转调度
 - ▣ 操作系统概念 (英文) ——5-Process Scheduling-例题与作业-23-v2.0

Multilevel Queue

■ Principles

two-level scheduling as an example

1. 对进程分门别类，不同类型进程位于不同队列，具有不同的调度机会/优先级
2. 两级调度：队间调度 + 队内调度

- the ready queue is partitioned into separate queues, for example

- foreground (interactive)
- background (batch)

- each queue takes its own scheduling algorithm,

- foreground – RR
- background – FCFS

- scheduling is done among the queues at first, the selected queue is then scheduled among its processes

■ E.g. O(1) scheduling in Linux

- refer to Appendix 5-3 Scheduling in Linux ▶

Multilevel Queue

■ Implementation

- ❑ 1. fixed priority scheduling for scheduling among queues
 - e.g., serve all from foreground then from background)
 - possibility of starvation
- ❑ 2. time slice based scheduling
 - each queue gets a certain amount of CPU time which it can schedule amongst its processes
 - e.g. , 80% to foreground in RR, 20% to background in FCFS

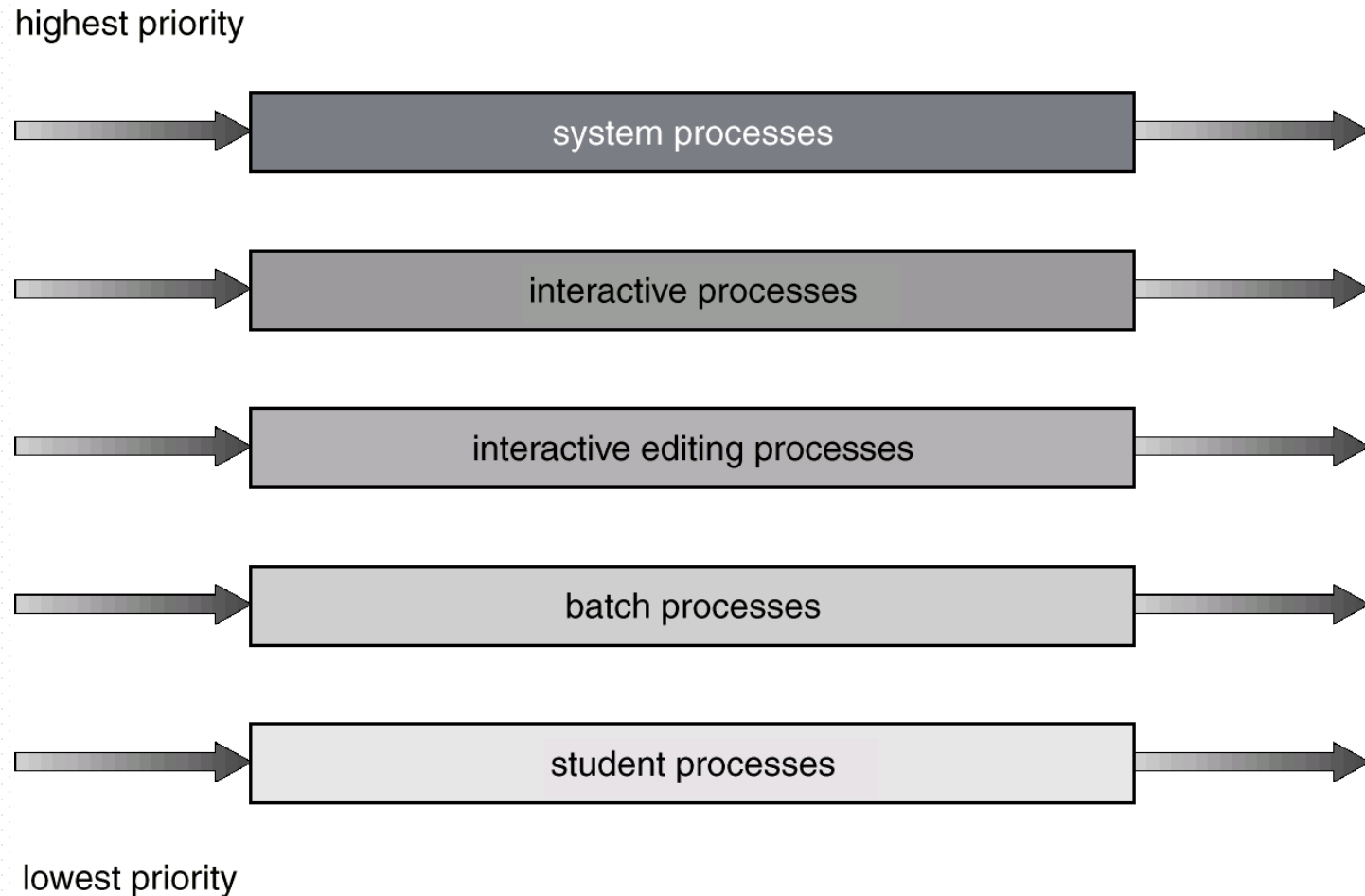


Fig. 5.6 Multilevel Queue Scheduling

Multilevel Queue

- The two-level queue scheduling scheme is somewhat similar to that in
 - ▣ user thread scheduling in 5.4
 - ▣ multiple-processor scheduling and multithreaded multicore CPU scheduling in 5.5

Multilevel Feedback Queue

■ Principles

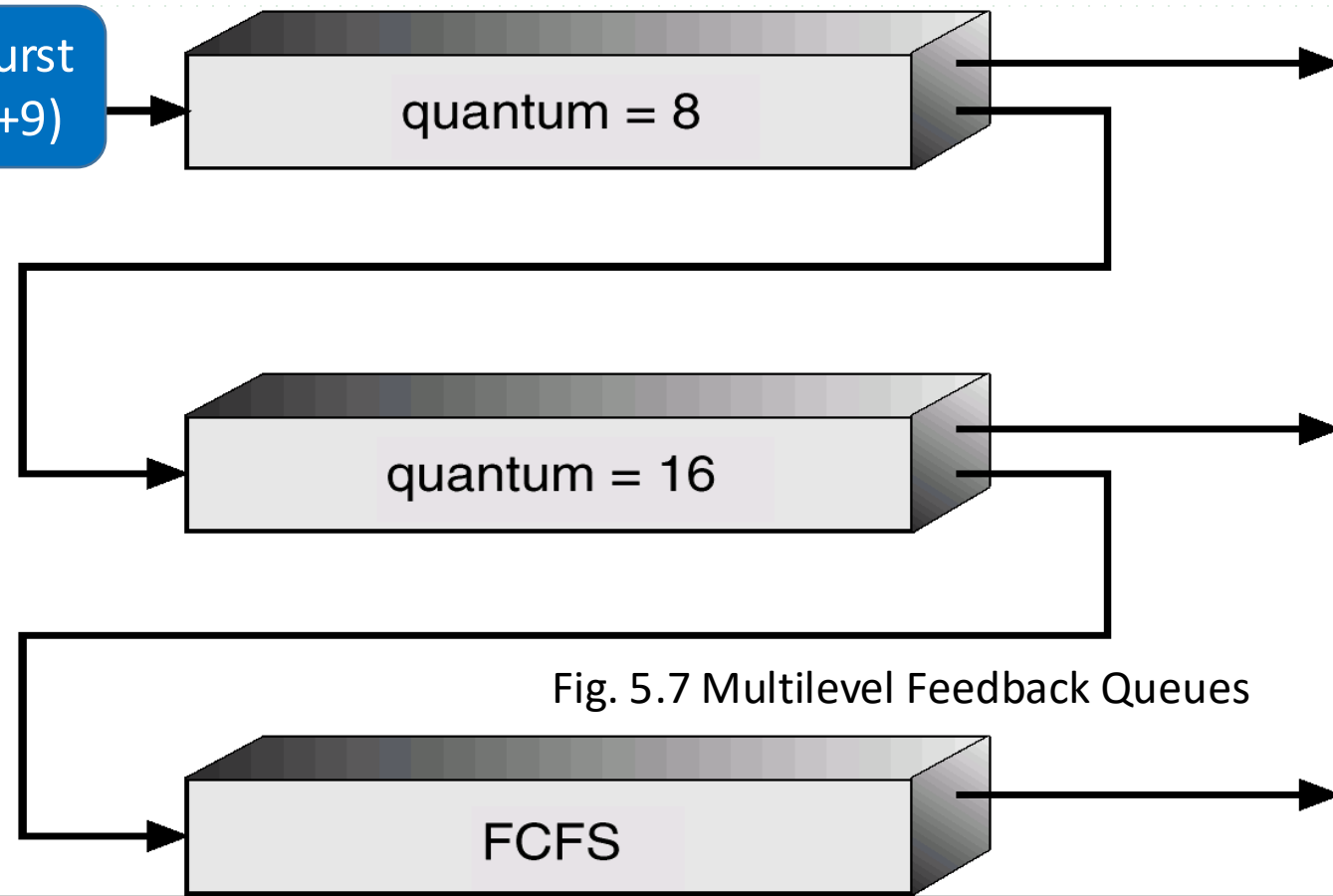
- ❑ a process can move between the various queues, and aging can be implemented in this way
- ❑ multilevel-feedback-queue scheduler defined by the following parameters
 - the number of queues
 - scheduling algorithms for each queue
 - the method used to determine when to **upgrade/升级** a process
 - the method used to determine when to **demote/降级** a process
 - the method used to determine which queue a process will enter
 - when that process needs service

Example 1

a process with burst length of (8+16+9)

■ Three queues

- ❑ Q_0 – time quantum 8 milliseconds
- ❑ Q_1 – time quantum 16 milliseconds
- ❑ Q_2 – FCFS

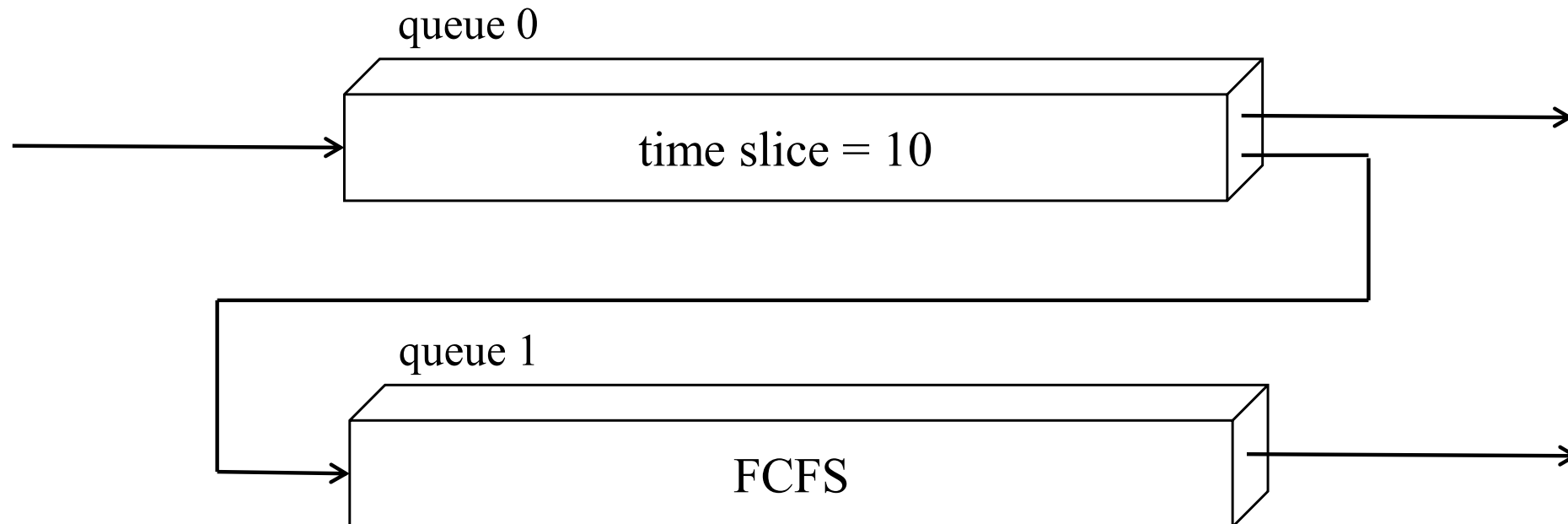


■ Scheduling

- ❑ a new job enters queue Q_0 which is served **FCFS**. When it gains CPU, the job receives 8 milliseconds. If it does not finish in 8 milliseconds, the job is moved to queue Q_1
- ❑ at Q_1 , job is again served **FCFS** and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2

Example 2

- As shown below, OS takes a two-level feedback-queue scheme to allocate CPU for concurrent processes
- A process entering the system is at first put in queue 0, and sequentially given a CPU time slice of 10 milliseconds. If it does not finish within this time, it is moved to the tail of queue 1. Processes in queue 1 run on FCFS scheduling, but are permitted to run only when there is no process in queue 0. When a process P_i in queue 1 is running on CPU and a new process P_j enters the system, P_j will preempt the CPU occupied by P_i



Example 2

- Consider the processes P_0, P_1, P_2, P_3 . For $0 \leq i \leq 3$, the arrival time, the length of the CPU burst time, and the priority of each P_i are given as below

Process	Arrival time	Burst Time	Priority
P_0	0.0	6.0	10
P_1	4.0	15.0	8
P_2	8.0	4.0	6
P_3	12.0	13.0	4

无用、
干扰信息

- For the snapshot shown above, suppose that two-level feedback-queue scheduling is employed (Q_0 – time quantum 4 milliseconds, Q_1 – FCFS)
 - (1) Draw the Gantt chart that illustrates the execution of these processes.
 - (2) What are the turnaround times for the four processes?

Highest Response Rate First (HRRF)

- HRRF

- ▣ Highest Response Rate First, 响应比最高者优先

- 引入

- ▣ FCFS: 只考虑作业等待时间, 利于减少waiting time
 - ▣ SJF: 只考虑估计的 (未来) 作业运行时间(burst time)

将上述2算法结合, 既照顾短作业, 又不会使长作业长时间等待, 改善调度性能

- 响应时间 = 进程进入系统后的等待时间(in ready state) + 估计计算时间(CPU burst)

- 响应比 = 响应时间/估计计算时间
= 1 + 等待时间/估计计算时间

HRRF算法中定义的响应时间, 与5.2节调度准则中定义的响应时间response time (专门针对第一章提到的交互式系统) 不一样, 各自有其适用范围

Highest Response Rate First (HRRF)

■ HRRF调度算法

- 从就绪队列中选择响应比最高的进程，分配CPU
- 非抢占式

■ 特点

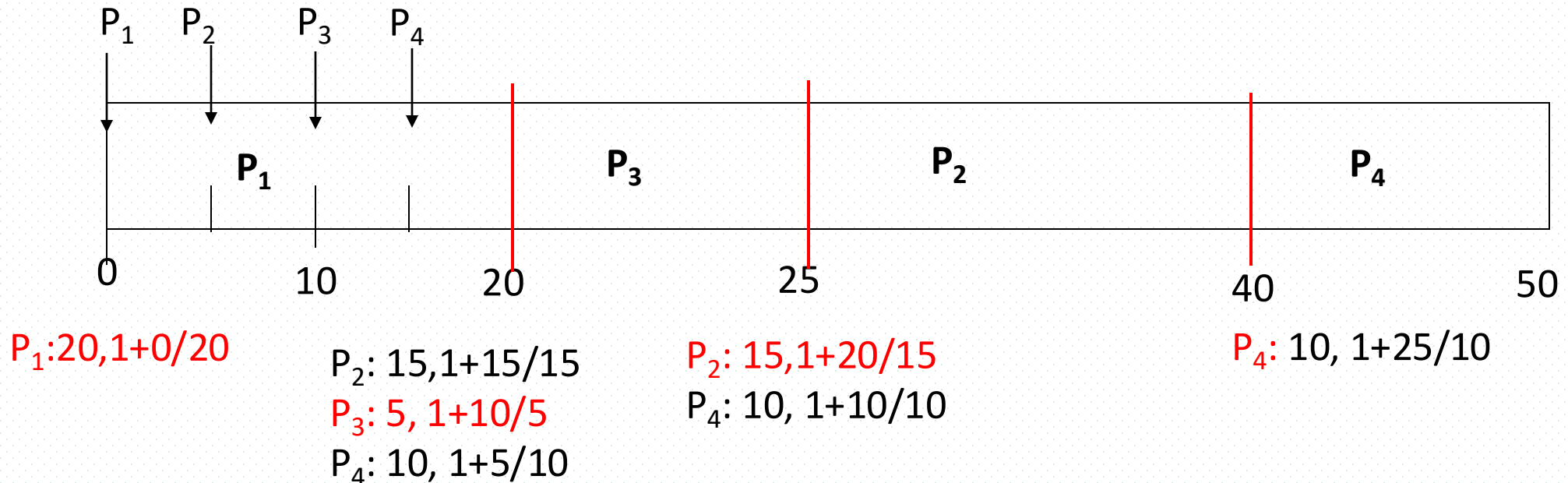
- 短作业的估计计算时间短，响应比较高，因此本算法优待(prefer to)短作业
- 对于长作业，如果在系统中等待时间足够长，将获得很高的响应比，进而被优先调度，不会发生“饿死”现象

- $SJF < HRRF \text{ 的平均周转时间} < FCFS$

Example

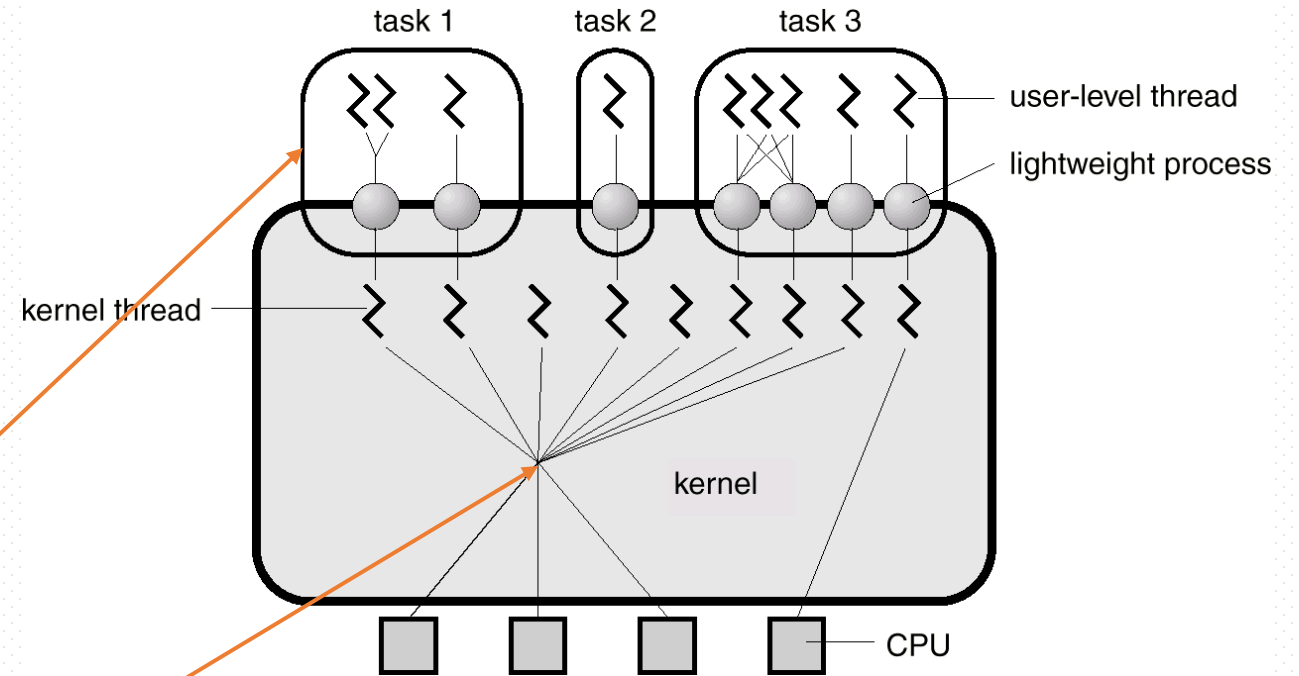
Process	Arrival Time	Burst Time
P ₁	0.0	20
P ₂	5.0	15
P ₃	10.0	5
P ₄	15.0	10

- 开始时, P₁, P₁被选中, 执行时间20
- P₁执行完后, P₂、P₃、P₄的响应比分别为
 $1 + 15/15$ 、 $1 + 10/5$ 、 $1 + 5/10$
 , P₃被选中, 执行时间5
- P₃执行完后, P₂、P₄的响应比分别为
 $1 + 20/15$ 、 $1 + 10/10$
 , P₂被选中, 执行时间15
- P₂执行完后, P₄执行, 执行时间10



5.4 Thread Scheduling

- Kernel-level threads are managed by OS, while user-level threads by thread library
- How to conduct scheduling involving KLT and ULT



- Two-level scheduling for user kernels
 - ▣ local scheduling: the **threads library** decides which user threads in processes to put onto an available LWP, being mapped into a kernel thread
 - many-to-one, one-to-one, many-to-many
 - ▣ global scheduling: the **OS kernel** decides which kernel thread is allocated CPU to run next
- Somewhat similar to three-level-queue scheduling

例9 用户级线程调度

Thread Scheduling in Pthread: Contention Scope(竞争范围)

PCS, SCS:
concepts in Pthread

■ Process-contention scope (PCS) 【进程竞争范围, local scheduling】

- ❑ for many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP
 - since scheduling competition is within the process
 - /* 决定将应用进程内的哪个线程安排在LWP, 参与后续调度
- ❑ the number of LWPs is controlled by thread library
- ❑ typically done via priority set by programmer, priority-based preemption is allowed
 - 采用优先级调度, 由编程者设定线程优先级; 允许高优先级线程抢占低优先级线程

Java19虚拟线程

■ System-contention scope (SCS) 【系统竞争范围, global scheduling】

- ❑ OS schedules kernel threads onto available CPU
- ❑ competition among all kernel threads in system
- ❑ for one-to-one mapping in Windows, Linux and Solaris, SCS scheduling is used

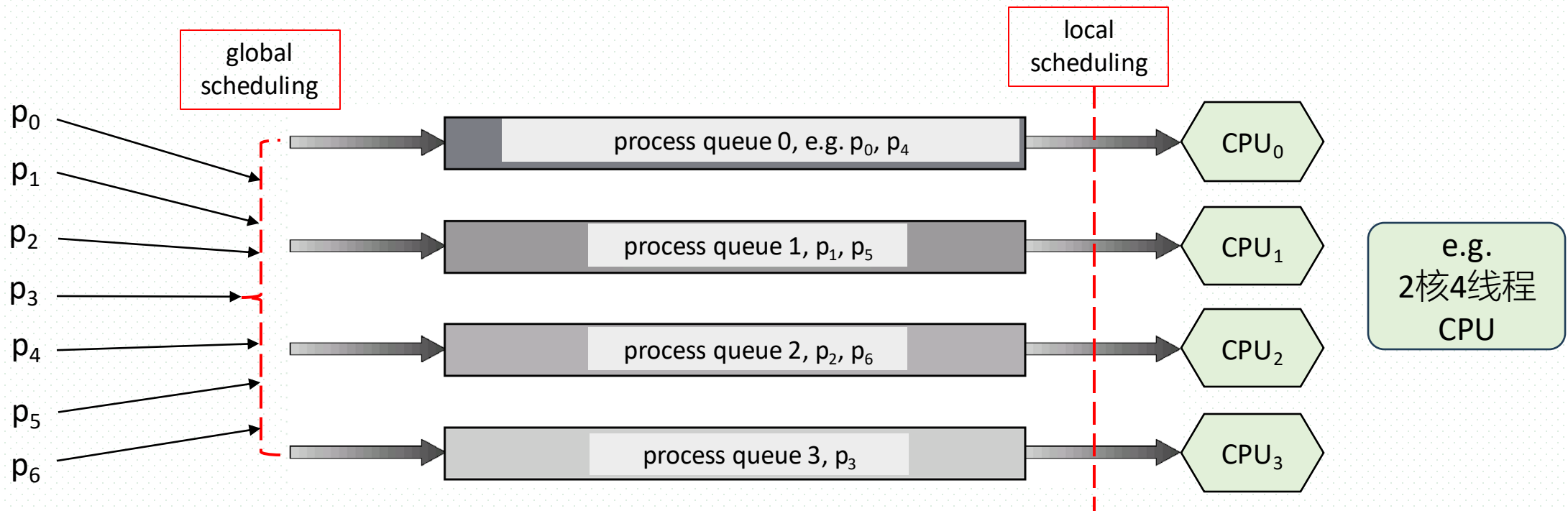
Pthread Scheduling

- Pthread_create() API allows specifying either PCS or SCS during thread creation
 - ▣ PTHREAD_SCOPE_PROCESS: schedules threads using PCS scheduling
 - ▣ PTHREAD_SCOPE_SYSTEM: schedules threads using SCS scheduling
- On system implementing **many-to-many** model, PCS policy schedules user-level threads on to available LWPs, the number of LWPs is maintained by the thread library
 - ▣ PCS policy creates and binds a LWP for each user-level thread, mapping thread using **one-to-one** policy
- Getting and setting contention scope policy in Pthread IPC

```
pthread_attr_setscope(pthread_attr_t *attr, int scope)
pthread_attr_getscope(pthread_attr_t *attr, int scope)
```

5.5 Multiple-Processor Scheduling

- Two-level scheduling in multi-processor system
 - ▣ Step 1. global scheduling: assign arriving processes to different CPU/processor queues
 - which CPU/processor can be allocated to the selected processes?
 - ▣ Step 2. local scheduling: for each CPU/processor, schedule processes in a queue to run
- Somewhat similar to two-level-queue scheduling



Multiple-Processor Scheduling

- Concerns in multiprocessor scheduling
 - ▣ Multiple-processor scheduling approaches
 - homogeneous processors, symmetric/asymmetric multiprocessing
 - ▣ Processor affinity
 - ▣ Load balancing
 - ▣ Multicore processors and Multithreading
 - 多线程化/超线程技术,
 - 通过虚拟化, 1个物理CPU上构造出多个作为可调度对象的逻辑CPU

Multiple-processor Scheduling Approaches

- CPU scheduling is more complex when multiple CPUs are available
 - ▣ Homogeneous/异构processors within a multiprocessor system
 - 需要考虑processor是否适合承担任务
 - e.g. for a gaming task, it should be assigned to a CPU with integrated graphics card, such as intel i9xxx processor, not E5 processor without graphics card
 - ▣ Asymmetric/非对称 multiprocessing
 - one processor makes scheduling decisions, does I/O operations and accesses the system data structures (管理节点, master) , the other processors execute user programs (执行节点, slave)
 - ▣ Symmetric multiprocessing(SMP)
 - each processor is self-scheduling, all processes in common ready queue, or each has its own private queue of ready processes
 - 每个process的角色、功能一样, 无差别
 - currently, most common

CPU/Processor Affinity

- Processor affinity (亲和)

将进程绑定/运行在特定processor, 或特定的CPU core上

- because of high scheduling cost, avoid migration of processes from one processor to another, and instead attempt to keep a processor running on the same processor
- process has affinity for processor on which it is currently running

- Hard affinity

- alleviate **system calls** to specify a subset of processors on which the process may run
 - 指定进程运行的处理器

- Soft affinity

- OS attempt to keep a process on a single processor, but it is possible for the process to migrate between processors

- Linux scheduler implements **soft affinity**, it also provides system call **sched_setaffinity()** to support hard affinity

Processor Affinity in Linux

- 在 Linux 系统中，任务通过设置 `cpu_set_t` 掩码表示可以执行任务（以线程为单位）的 CPU 核心(core)集合，掩码中的每一位都对应于一个 CPU
- Linux 提供了一系列对 `cpu_set_t` 进行操作的宏

Linux 提供的操作 CPU 核心集合掩码的宏

```
# include < sched . h >
//对 set 进行初始化， 设置为空
void CPU_ZERO (cpu_set_t *set );
//将对应 CPU 核心加入set中
void CPU_SET (int cpu , cpu_set_t *set );
// 将对应 CPU 核心从 set 中删除
void CPU_CLR (int cpu, cpu_set_t *set );
//判断对应CPU 核心是否在set中， 是则返回1； 否则返回0
int CPU_ISSET (int cpu, cpu_set_t *set );
//返回当前set中的 CPU 核心数量
int CPU_COUNT(cpu_set_t *set );
```

Processor Affinity in Linux

- 在这些宏操作的基础上，Linux 提供了设置和获取任务亲和性的系统调用
 - ▣ 进程可以通过 `sched_setaffinity` 设置任务对于 CPU 核心的亲和性，或者通过 `sched_getaffinity` 获取任务的亲和性配置
 - ▣ 由于线程是 Linux 的调度单元，此处的 `pid` 参数实际上是 Linux 为每个线程分配的标识符，如果 `pid` 为 0，则代表操作对象是当前的调用者线程

Linux 的任务/线程亲和性接口

```
# include < sched.h >

int sched_setaffinity ( pid_t pid , size_t cpusetsize , const cp_set_t *mask );

int sched_getaffinity ( pid_t pid , size_t cpusetsize , cpu_set_t *mask );
```

Load Balancing in Multiple-processor/Distributed Systems

- If SMP, need to keep all CPUs loaded for efficiency
- Load balancing attempts to keep workload evenly distributed
 - ▣ 各CPU上运行的进程数目、计算负荷尽可能均衡，不要相差过大
- Push migration
 - ▣ periodic task checks load on each processor, and if found workload is not evenly distributed, e.g. workload on a CPU is heavy, pushes the tasks from the overloaded CPU to other CPUs
- Pull migration
 - ▣ idle processors pulls waiting task from busy processor

Multithreaded Multicore Processor

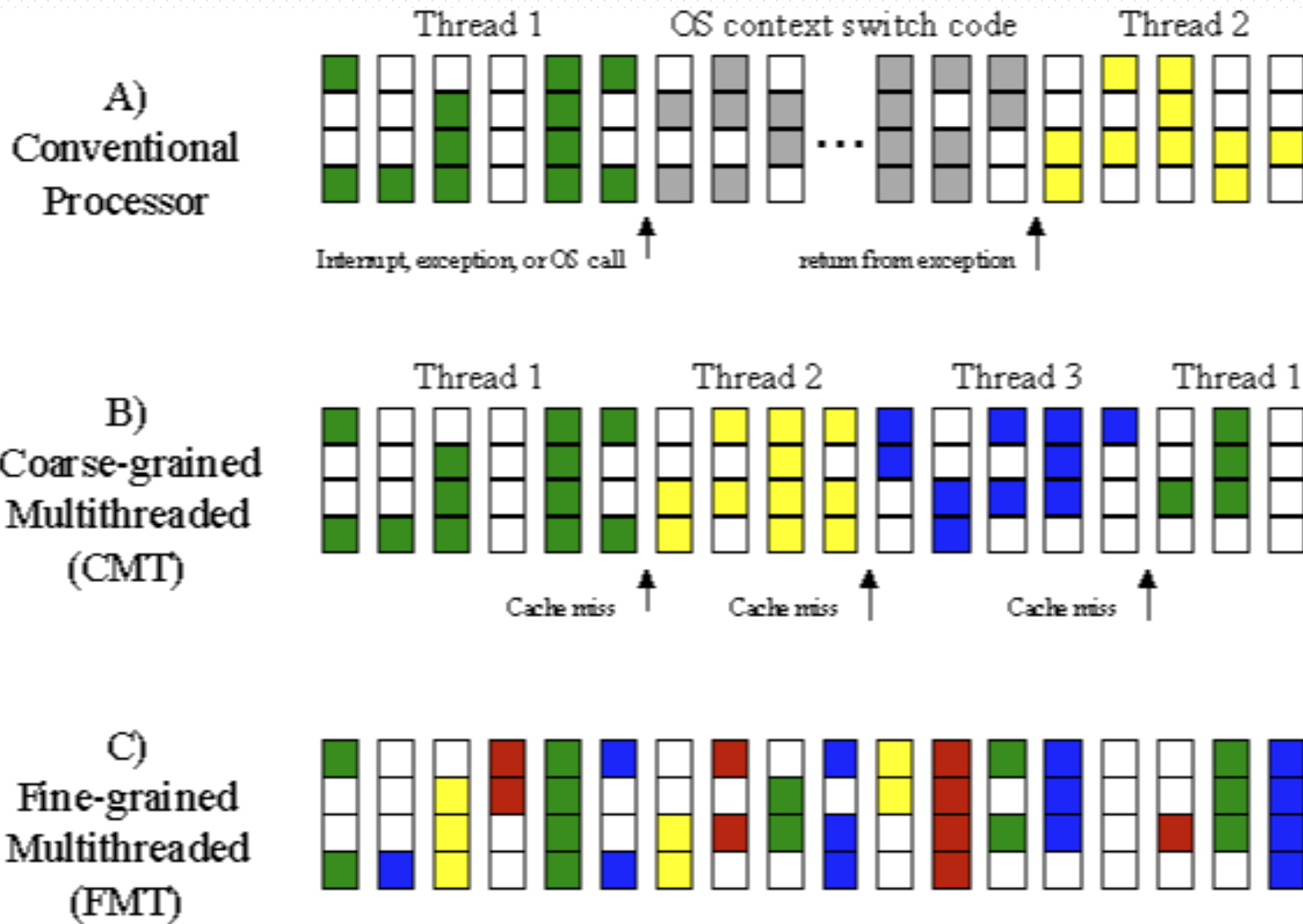
- Multiple threads per core—hardware thread, hyperthread/超线程
 - ▣ core with **hardware thread**, e.g. with **hyperthread HT**
 - ▣ takes advantage of **memory stall (内存停顿)** to make progress on another thread while memory retrieve happens
 - 一个物理核上运行多个**硬件线程**（**超线程HT?**），每个**硬件线程**类似于一个独立的**逻辑处理器**
 - 两个**硬件线程/逻辑处理器**，利用**内存停顿**，交错执行
 - ▣ UltraSPARC T3 CPU
 - 16 cores, 8 hard threads per core, thus **128** logical CPU
 - ▣ 从操作系统角度，每个**硬件线程/逻辑处理器**都是一个可以调度分配的CPU

Two Types of Multithreading in a Core

- Fine-grained multithreading/细粒度多线程
 - ▣ hardware thread switch at the boundary of an **instruction cycle/指令周期**
 - ▣ e.g. refer to the example in 

- Coarse-grained multithreading/粗粒度多线程
 - ▣ hardware thread switch due to a long-latency event, such as memory stall/内存停顿
 - ▣ e.g. C and M in the example 

Multithreaded Multicore Systems



Scheduling in multithreaded multicore

- Two levels of scheduling
 - ▣ OS level
 - OS choose which software thread (e.g. 应用线程、程序、进程) to run on each hardware thread (logical processor) on CPU
 - ▣ **CPU level**
 - CPU core decides which hardware thread to run
 - /*哪个逻辑CPU真正占用CPU core的计算/物理资源去执行
- Somewhat similar to two-level-queue scheduling

Scheduling in multithreaded multicore

- Example1. Scheduling policy at CPU level in UltraSparc T3
 - round- robin algorithm 【时间片轮转】
 - schedule the eight hardware threads to each core
- Example2. Scheduling policy at CPU level in Intel Itanium dual core CPU
 - each hardware thread is assigned a dynamic urgency value ranging from 0 to 7, with 0 representing the lowest urgency and 7 the highest
 - 优先级调度, 优先级=urgency value
 - the Itanium identifies five different events that may trigger a thread switch
 - when one of these events occurs, the thread-switching logic compares the urgency of the two threads and selects the thread with the highest urgency value to execute on the processor core

补充资料：Types of Hardware Multithreading in CPU Core

- Objects of multithreading
 - ▣ 同一CPU流水线上执行多个线程，提高（取指、译码、计算、访存等）流水线器件利用率
- 三种实现方式
 - ▣ Interleaved/交织 multithreading
 - ▣ Block multithreading
 - ▣ Simultaneous multithreading

补充资料：Types of Hardware Multithreading in CPU Core

- Interleaved multithreading
 - 细粒度，前述Fine-grained multithreading
 - 线程context切换发生在每一个boundary of an instruction cycle，每个周期切换一次，但保留流水线信息
- Block multithreading
 - 粗粒度，前述coarse-grained multithreading
 - 当线程执行发生了latency较高的事件，如memory stall，发起线程切换，并清除流水线中的信息，执行其它线程来隐藏降低latency
- Simultaneous multithreading
 - Intel CPU中的超线程HyperThreading
 - 通过增加CPU计算资源，如取指单元、译码单元、Issue队列、CDB、BTB、分支预测器，以及寄存器等部件数量，提高运算单元利用率

5.6 Scheduling in Real-time Systems

■ Real-time system

- ❑ a computer and/or software system that reacts to **external events** in limited time intervals (**deadline**, 截止期) before the events become obsolete (失效)
- ❑ specific-purpose systems, such as industrial control systems
- ❑ each external event corresponds to critical task or process in the system, that is, for each external event, there is a critical task or process to react to or handle it

■ Soft real-time systems

- ❑ no guarantee as to when critical real-time process will be scheduled

■ Hard real-time systems

- ❑ task must be serviced by its deadline

Example

- The task or process **deadline** in real time systems
 - ▣ the time limit before which the task/process should be selected by scheduler, run on CPU and complete processing, in order to react to and handle the external event in time

<u>Events</u>	<u>Critical process</u>	<u>Arrival Time</u>	<u>burst time</u>	<u>deadline</u>	<u>priority number</u>
1	P ₁	0.0	4.0	7.00	3
2	P ₂	3.0	2.0	5.50	1
3	P ₃	4.0	2.0	12.01	4
4	P ₄	6.0	4.0	11.00	2

Note: a small priority number means higher priority

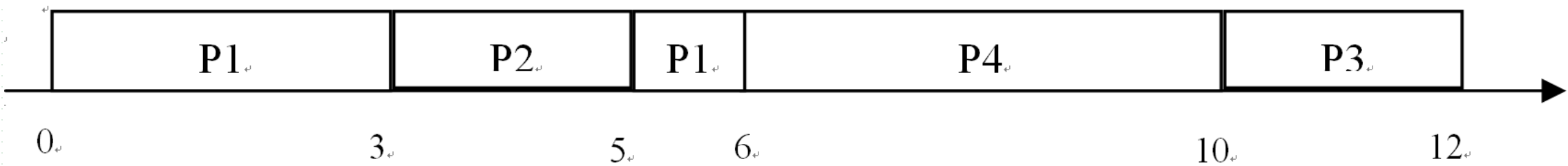


Example

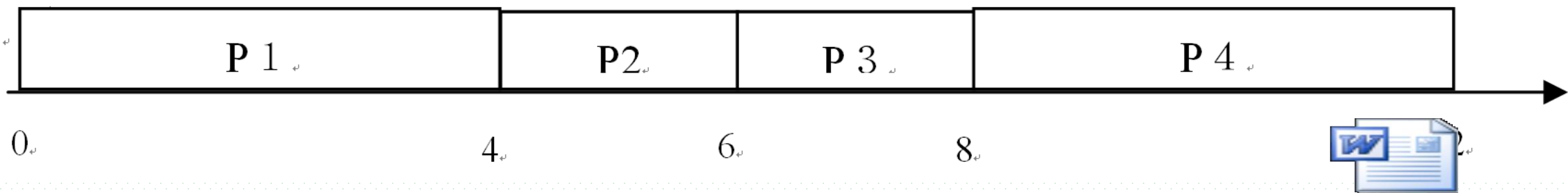
- Suppose that FCFS scheduling is employed,
 - give the Gantt charts illustrating the execution of these processes
 - give the turnaround time and waiting time of each process
 - give the average turnaround time and the waiting time
 - which event will be treated with in time, and which not ?
 - be treated with in time:
the event will not finish after its deadline
- Suppose that priority-based preemptive scheduling scheduling is employed,
 - give the Gantt charts illustrating the execution of these processes
 - give the turnaround time and waiting time of each process
 - give the average turnaround time and the waiting time
 - which event will be treated with in time, and which not ?

Example

- Priority-based preemptive
 - ▣ all processes are treated with in time



- FCFS
 - ▣ e1, e3 are treated with in time



deadline到达时，如果进程没有结束，应继续执行，不会被强行终止



!!
μäΘÍÀýìâ-μÚ5Õ

Scheduling Objective: Minimizing Scheduling Latency

- When an event occurs, the system must respond to and service it as quickly as possible.
- Real-time system is **event-driven** and **time-critical/时间紧迫** system
 - external events → interrupts → scheduling among critical processes → the critical process responsible for reacting to and processing these events in limited time (before the deadline)
- Event latency
 - the amount of time that elapses from when an event occurs to when it is serviced
 - similar to **response time**

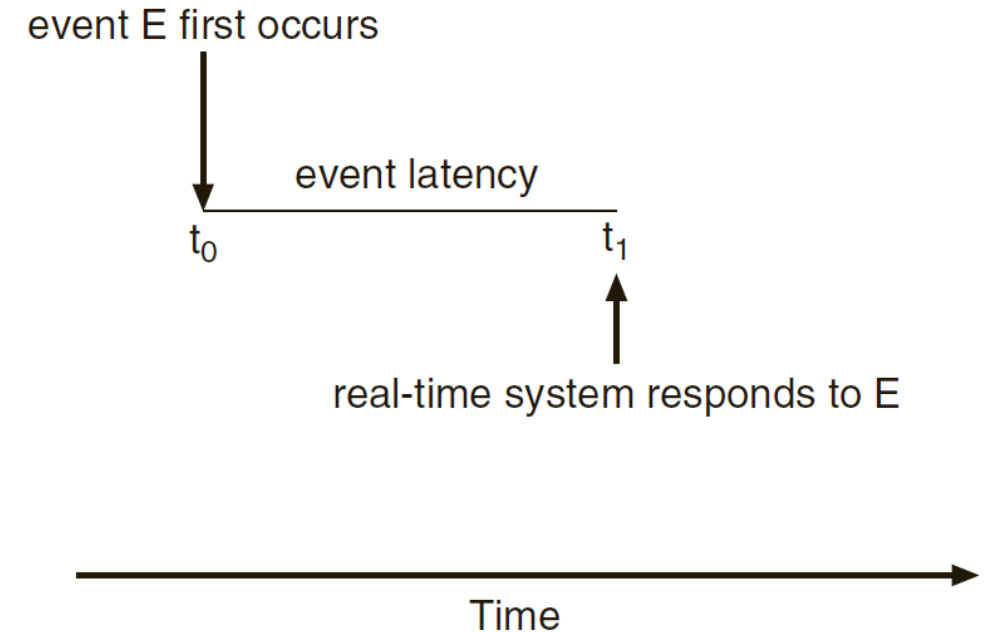


Figure 5.12 Event latency.

Minimizing Scheduling Latency

- Two types of latencies affect performance
 - ▣ interrupt latency/中断延迟
 - time from arrival of interrupt to start of routine that services interrupt
 - ▣ dispatch latency/调度延迟
 - time for schedule to take current process off CPU and switch to another

Event latency ??

= interrupt processing + dispatch latency + process execution
= interrupt latency + ISR + conflict + dispatch + process execution

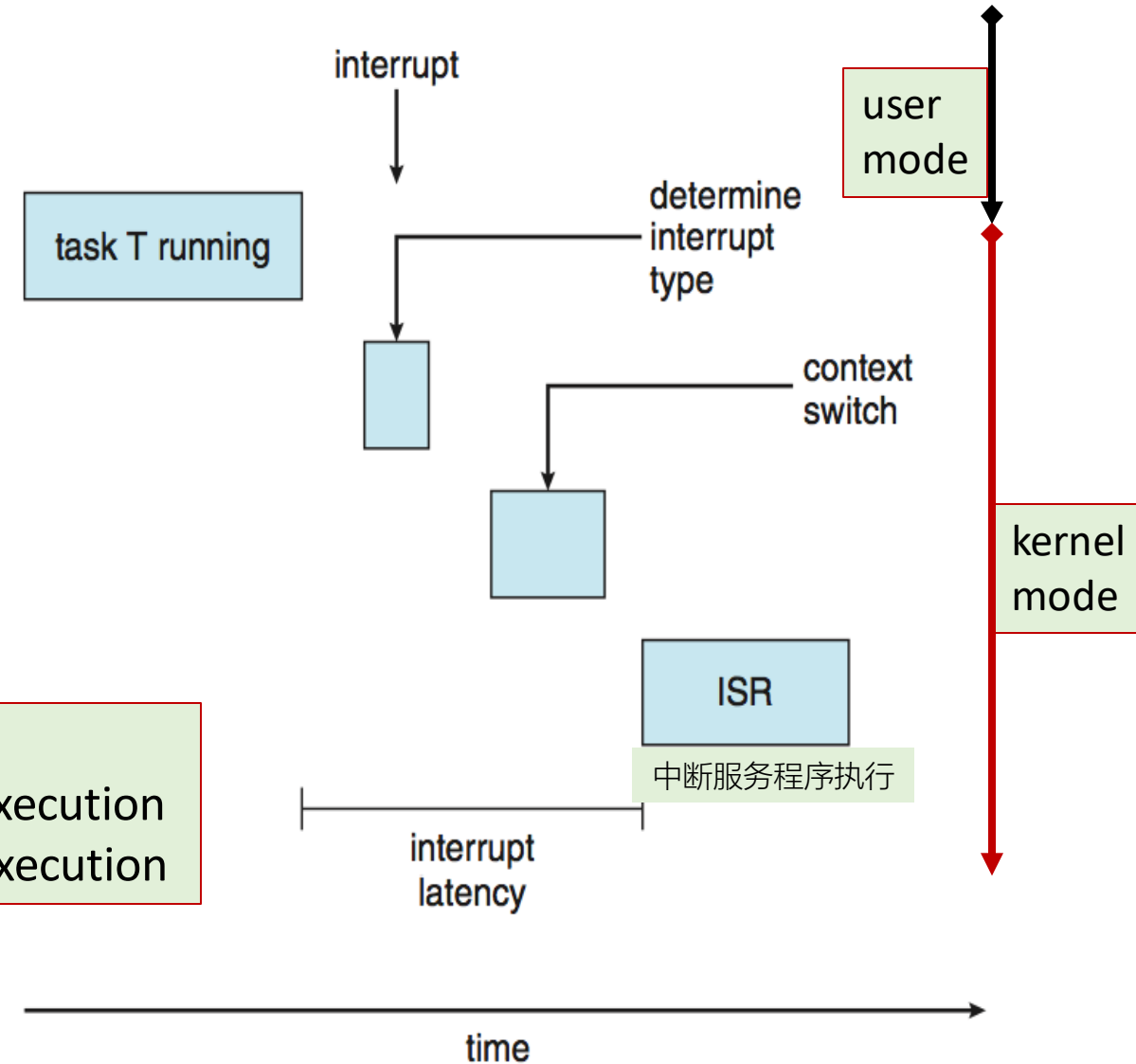


Figure 5.13 Interrupt latency

Minimizing Scheduling Latency

■ Dispatch latency consists of two phases

▣ conflict phase

- 1. preemption of any process running in kernel mode
- 2. releasing the resources occupied by low-priority processes (e.g. P_1) and needed by the high-priority process (e.g. P_2) in the above-mentioned example

▣ dispatch phase

- allocate resources for the selected high-priority process (e.g. P_2) to react to the external event and start the process' running

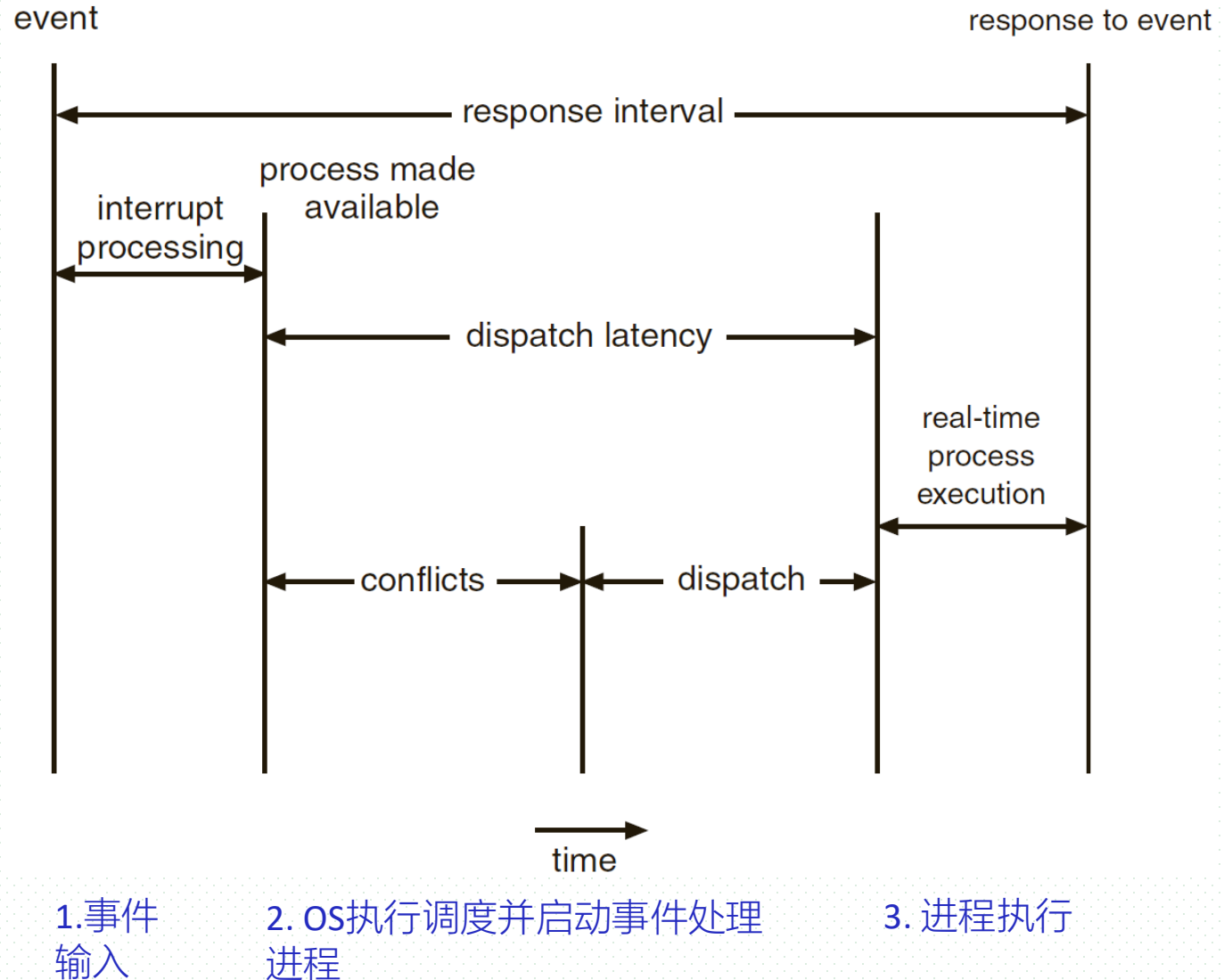


Figure 5.14 Dispatch latency

Priority-based Scheduling

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
 - ▣ but only guarantees soft real-time
- Scheduling in soft real-time system
 - ▣ all external requests are admitted to enter the system
 - ▣ the real-time processes have highest priorities, and do not degrade over time
 - ▣ the **dispatch latency** must be smaller, in order for a real-time process to start executing as faster as possible, once it is runnable. For this purpose, system calls should be allowed to be preemptive

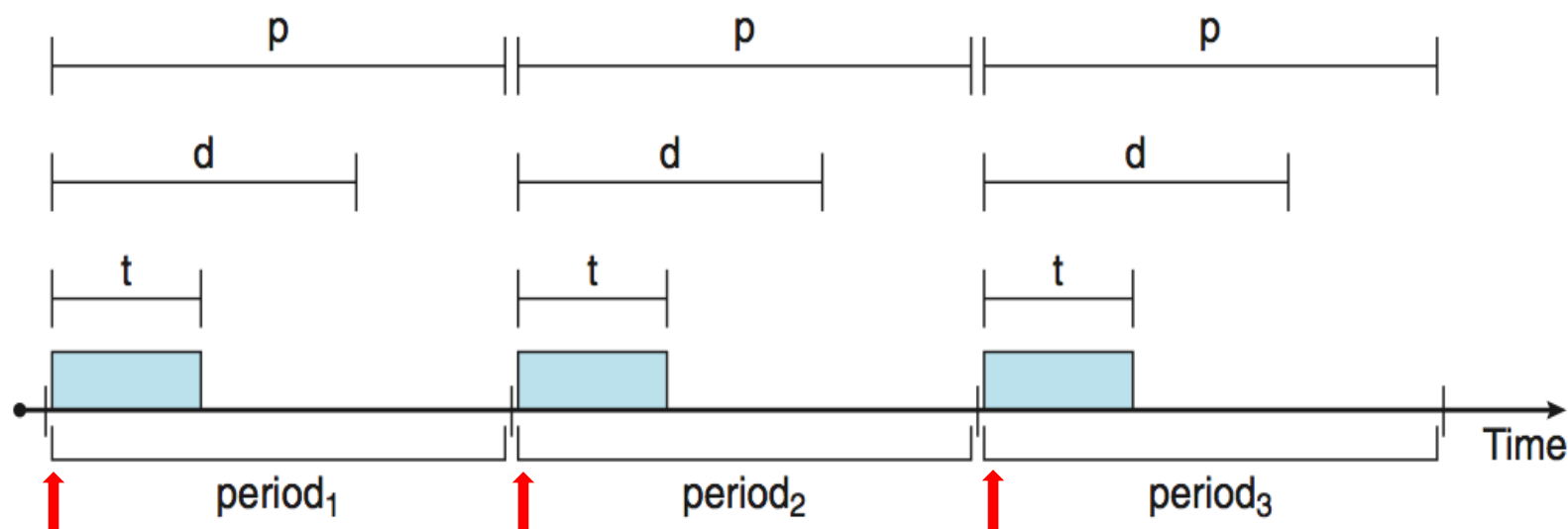
Priority-based Scheduling

- For hard real-time, scheduler must also provide ability to meet deadlines
- Scheduling in **hard** real-time system
 - **priority-based and preemptive scheduling**
 - the scheduler should know exactly about the critical tasks, such as arrival time, burst time, deadline and priority
 - on the basis of **schedulability analysis** (可调度性分析), enough resources are configured in systems, making that when the scheduler admits the critical processes to response to external events, these processes are guaranteed to complete before their deadlines
 - this is known as **resource reservation/资源预留**
- Another Example: **deterministic network**, 确定性网络
 - 带宽、延迟必须保证
 - 用于工业互联网等场景
 - 方法：在网络协议层2提供保障机制

Priority-based Scheduling

- Processes have new characteristics
 - ▣ **periodic** processes require CPU at constant intervals
 - 外部事件周期性到达, OS周期性启动处理进程
 - * e.g. 工业控制系统中的周期性温度巡检、压力巡检
 - ▣ has processing time t , deadline d , period p
 - ▣ $0 \leq t \leq d \leq p$,
 - e.g. $p=60s$, $t=100ms$
 $d=150ms$
 - ▣ **rate** of periodic task is $1/p$
 - 任务到达速率

↑: arriving



Scheduling policies for tasks in real-time system

- Scheduling objectives
 - ▣ guaranteeing that when a task enters the system, it will complete its burst no later than its deadline
 - 完成时间不晚于deadline
- CPU utilization in real-time scheduling
 - ▣ the rate of CPU time running on the user mode to complete real time tasks
- Three scheduling policies
 - ▣ Rate Monotonic Scheduling/单调速率调度
 - for periodic tasks
 - ▣ Earliest Deadline First Scheduling (EDF)/最早截止期优先
 - ▣ Proportional Share Scheduling/比例分享调度

Rate Monotonic (RM, 速率单调) Scheduling

- For scheduling periodic tasks, static priority based
- Rate
 - task arriving rate, that is
 - the inverse of task period—任务周期的倒数, $1/p$
- A priority is assigned to a process/task based on the inverse of its period
 - shorter periods = higher priority
 - longer periods = lower priority
 - 周期 p 大, 发生频率小, 任务不紧迫, 调度的优先级低
- Process with higher priority is allowed to preempt the CPU allocate to the process with lower priority
- 性质
 - 在所有基于静态优先级的实时调度策略中, RM策略是最优的
 - 如果RM策略无法在截止期deadline之前完成一组任务, 则该组任务也无法被其它基于静态优先级的调度策略在截止期前完成

Rate Monotonic Scheduling

- Example: P_1 is assigned a higher priority than P_2
 - P_1 's period $p_1=50\text{ms}$, P_1 's processing time/CPU burst $t_1=20\text{ms}$
 - P_2 's period $p_2=100\text{ms}$, P_2 's processing time/CPU burst $t_2=35\text{ms}$
 - deadline for P_1 and P_2 : complete its CPU burst by the start of its next period
 - deadline $d = \text{period } p$

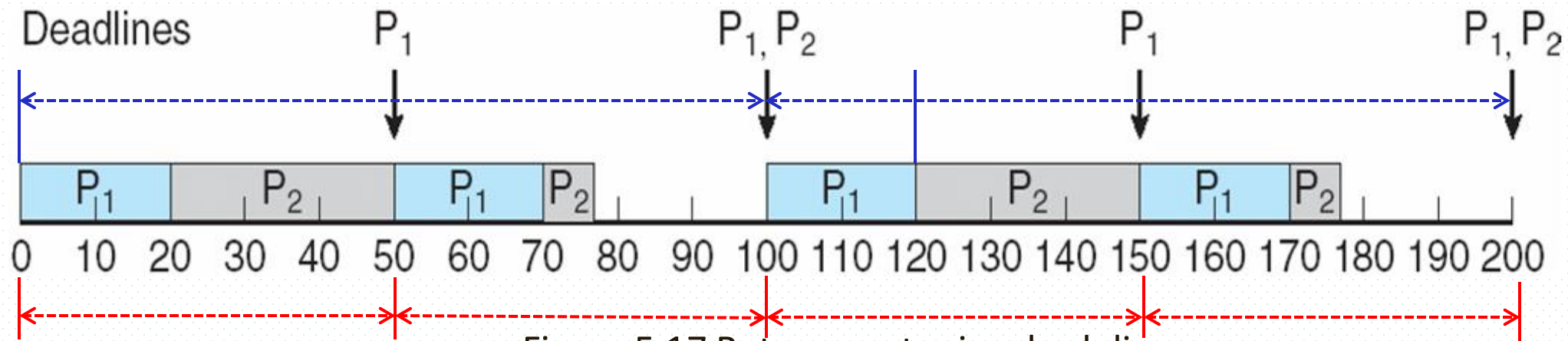


Figure 5.17 Rate-monotonic scheduling



$$\text{CPU utilization} = (20 + 30 + 20 + 5)/100 = 75\%$$

Rate Monotonic Scheduling

- Despite being optimal, then, rate-monotonic scheduling has a limitation
 - ▣ CPU utilization is bounded, and it is not always possible fully to maximize CPU resources
 - ▣ cannot guarantee that processes can be scheduled so that they meet their deadlines
- The **worst-case** (最小, 渐近下界 Ω) CPU utilization for scheduling N processes is

$$N(2^{1/N} - 1).$$

- ▣ $N=1$, utilization=100%
- ▣ $N=2$, bounded about 83%
- ▣ $N \rightarrow \infty$, utilization $\rightarrow 69\%$

Rate Monotonic Scheduling

- Rate monotonic scheduling **cannot** guarantee that processes can be scheduled so that they meet their deadlines
- Example
 - P_1 's period $p_1=50\text{ms}$, P_1 's processing time/CPU burst $t_1=25\text{ms}$
 - P_2 's period $p_2=80\text{ms}$, P_2 's processing time/CPU burst $t_2=35\text{ms}$
 - deadline for P_1 and P_2 : complete its CPU burst by the start of its next period

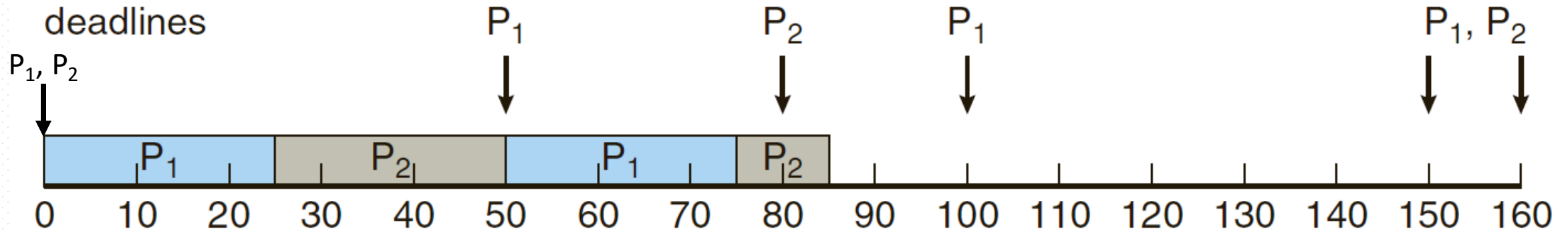


Figure 5.18 Missing deadlines
with rate-monotonic scheduling

原因：
在80ms的时段内， P_1 到达2次， P_2 到达1次；
前者总是抢占CPU优先运行；
 P_1 运行二轮、 P_2 运行一轮，共需 $25 + 25 + 35 = 85\text{ms}$

Earliest Deadline First Scheduling(EDF)

- Priorities are assigned to processes according to their deadlines
 - ▣ the earlier the deadline, the higher the priority
 - ▣ the later the deadline, the lower the priority /*截止期ddl越小, 任务越紧迫, 优先调度
- Under the EDF policy, when a process becomes runnable, it must announce its deadline requirements to the system. Priorities may have to be adjusted to reflect the deadline of the newly runnable process
 - ▣ dynamic priority
- Unlike the rate-monotonic algorithm, EDF scheduling does not require that processes be periodic, nor must a process require a constant amount of CPU time per burst
 - ▣ can be applied to non-period tasks
- The only requirement
 - ▣ a process announce its deadline to the scheduler when it becomes runnable

Earliest Deadline First Scheduling (EDF)

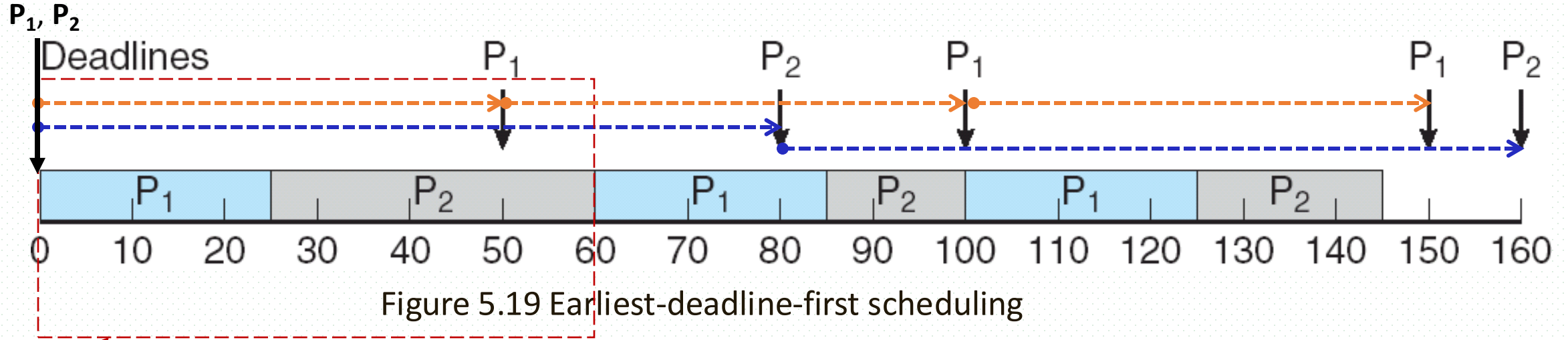
- EDF scheduling is **theoretically optimal**
 - ▣ theoretically, it can schedule processes so that
 - each process can meet its deadline requirements
 - CPU utilization will be 100 percent
- 问题：如果资源不够，如何保证？



Earliest Deadline First Scheduling(EDF)

■ Example

- P_1 's period $p_1=50\text{ms}$, P_1 's processing time/CPU burst $t_1=25\text{ms}$
- P_2 's period $p_2=80\text{ms}$, P_2 's processing time/CPU burst $t_2=35\text{ms}$
- deadline for P_1 and P_2 : complete its CPU burst by the start of its next period



- Step 1: Initially, P_1 is assigned an **initial higher priority** than P_2 , due to its shorter period/deadline 50ms, and is selected to run; and P_2 begins running at time 25, i.e. at the end of the CPU burst for P_1 , and then ends its first burst at time 60

Earliest Deadline First Scheduling(EDF)

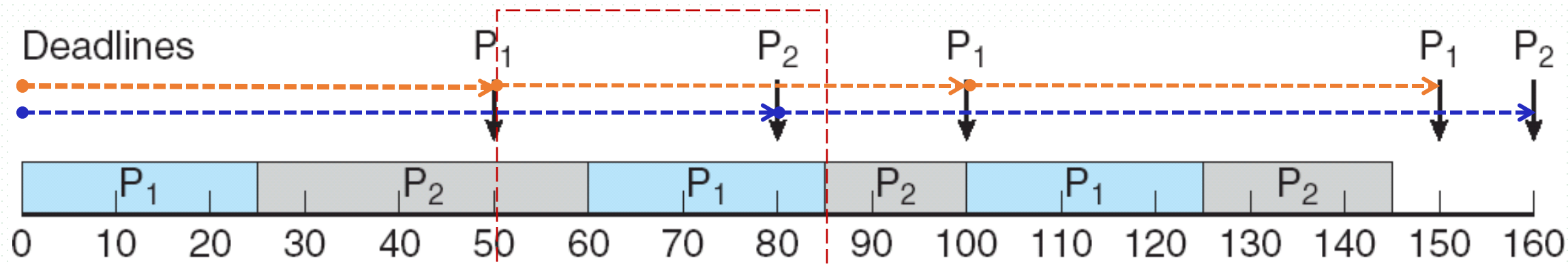
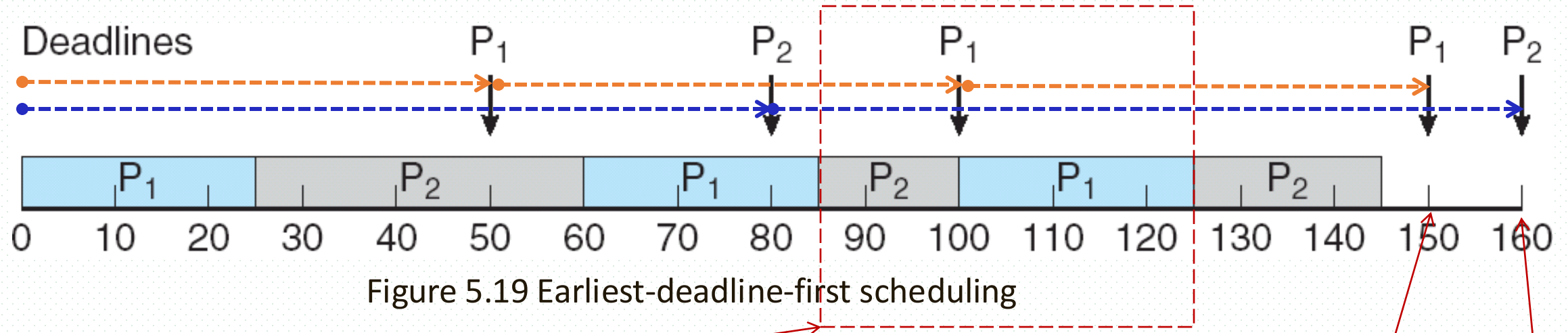


Figure 5.19 Earliest-deadline-first scheduling

- Step 2: At time 50, the P_1 's second burst comes, but EDF scheduling allows P_2 to continue running
 - ▣ P_2 now has a higher priority than P_1 , because its next deadline (at time 80, P_2 's second burst comes) is earlier than that of P_1 (at time 100, P_1 's third burst comes)
 - ▣ thus, both P_1 and P_2 meet their first deadlines
- Process P_1 again begins running at time 60 and completes its second CPU burst at time 85, also meeting its second deadline at time 100

Earliest Deadline First Scheduling(EDF)



- Step 3: At time 85, P_2 's second burst, which comes at time 80, begins running, only to be preempted by the third burst of P_1 at the start of its next period at time 100
 - ▣ at time 100, P_2 is preempted, because P_1 has an earlier deadline (time 150) than P_2 (time 160)

Proportional Share Scheduling(比例分享)

- CPU time is divide into T shares and are allocated among all processes in the system
- An application receives N shares where $N < T$, controlled by a admission controller (AC, 接纳控制器)
 - AC可以拒绝无法保证资源、deadline的processes
 - e.g. 电话打不通
- This ensures each application will receive N / T of the total processor time
- E.g. CPU time is divided into $T=100$ shares
 - process A, B, C and D sequentially need 50, 15, 20, and 30 shares, respectively
 - AC allocates 50, 15, and 20 shares of CPU time to process A, B and C, and rejects the request of 30 shares from process D
 - for A, B, C, $50+15+20=85 < T=100$, their requests are granted
 - for D, its request $30 > 100-(50+15+20)$, is rejected

Virtualization and Scheduling 【略】

- Virtualization software schedules multiple guests onto CPU(s)
- Each guest doing its own scheduling
 - ▣ Not knowing it does not own the CPUs
 - ▣ Can result in poor response time
 - ▣ Can effect time-of-day clocks in guests
- Can undo good scheduling algorithm efforts of guests

POSIX Real-Time Scheduling

- The POSIX.1b standard provides extensions for real-time computing
 - ▣ its API provides functions for managing real-time threads
- Defines two scheduling classes for real-time threads
 - ▣ SCHED_FIFO - threads are scheduled using a FCFS strategy with a FIFO queue. There is no time-slicing for threads of equal priority
 - ▣ SCHED_RR - similar to SCHED_FIFO except time-slicing occurs for threads of equal priority
- Defines two functions for getting and setting scheduling policy:
`pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
`pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`

5.7 Operating System Examples 【略】

- Linux scheduling
- Windows scheduling
- Solaris scheduling

Linux Scheduling 【略】

- Prior to kernel version 2.4, ran variation of standard UNIX scheduling algorithm
 - ▣ not consider SMP systems
 - ▣ refer to Appendix 5-2 Scheduling in Unix
- Version 2.4, $O(n)$ scheduler
- Version 2.5 moved to constant order $O(1)$ scheduler
- in Version 2.6.23 +, Completely Fair Scheduler (CFS)
- For more details, refer to Appendix 5-3 Scheduling in Linux

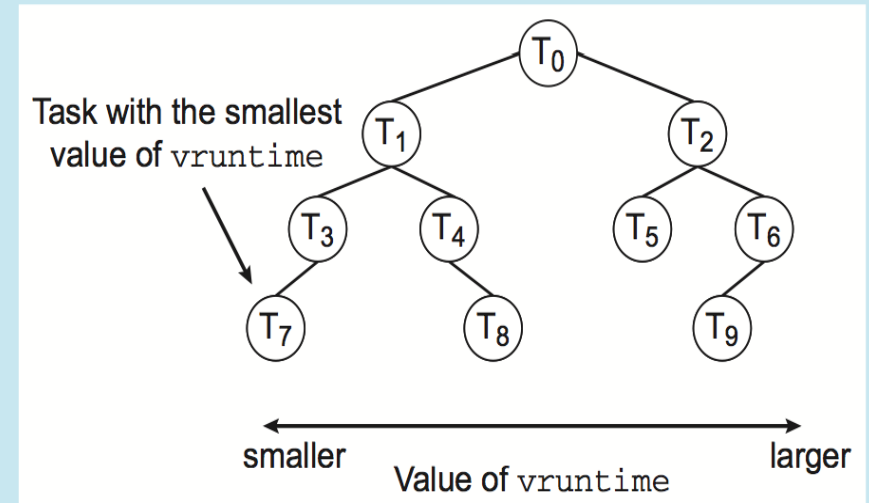
Linux Scheduling in Version 2.6.23 +

- **Completely Fair Scheduler** (CFS)
- **Scheduling classes**
 - Each has specific priority
 - Scheduler picks highest priority task in highest scheduling class
 - Rather than time quantum (时间片) based on fixed time allotments, based on proportion of CPU time
 - 可变长度时间片
 - 2 scheduling classes included, others can be added
 - default
 - real-time
- **Time quantum**s are calculated based on task **nice value** from -20 to +19
 - Lower value is higher priority
 - Calculates **target latency**
 - interval of time during which task should run at least once
 - Target latency can increase if say number of active tasks increases

Linux Scheduling in Version 2.6.23 +

- CFS scheduler maintains per task **virtual run time** (虚拟运行时间) in variable **vruntime**
 - ▣ associated with decay factor based on priority of task
 - lower priority is higher decay rate
 - ▣ normal default priority yields virtual run time = actual run time
- To decide next task to run, scheduler picks task with lowest virtual run time
 - ▣ 调度器选择虚拟时间最小的task

The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:

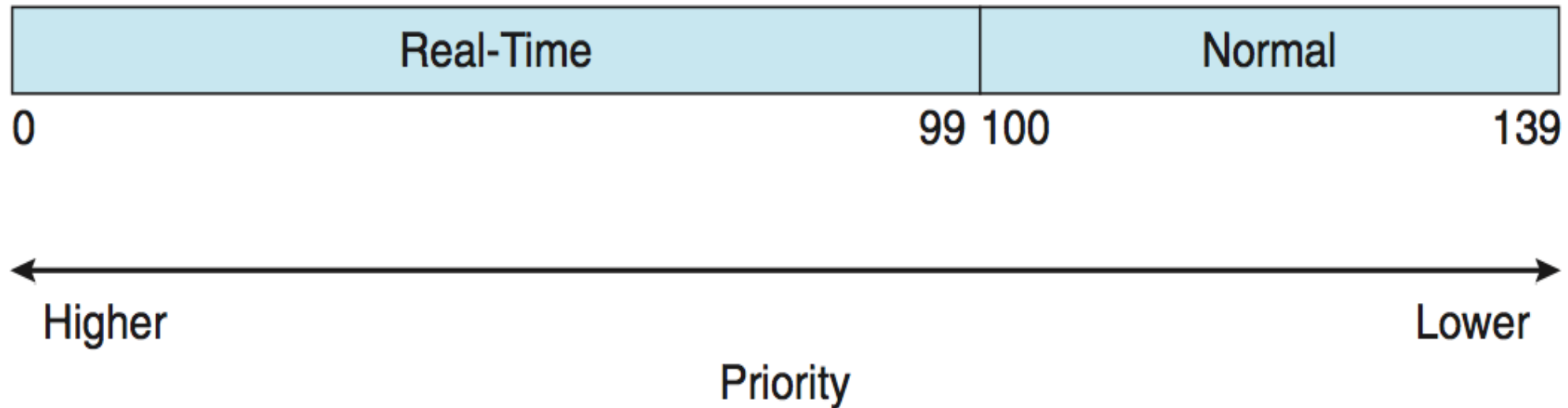


When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb.leftmost`, and thus determining which task to run next requires only retrieving the cached value.

CFS Performance

Linux Scheduling (Cont.)

- Real-time scheduling according to POSIX.1b
 - ▣ Real-time tasks have static priorities
- Real-time plus normal map into global priority scheme
- Nice value of -20 maps to global priority 100
- Nice value of +19 maps to priority 139



Priority in Linux Kernel

- 任务-调度类-调度器-调度策略/算法

调度类	调度器	调度策略/算法	描述及适用任务
实时调度类	DL	SCHED_DEADLINE	针对实时任务，类似EDF的调度策略，面向hard real-time
	RT	SCHED_FIFO	针对实时任务，执行直至结束，或被更高优先级任务抢占
	RT	SCHED_RR	针对实时任务，时间片轮转
非实时调度类	CFS	SCHED_OTHER/Normal	公平的分时调度策略
	CFS	SCHED_BATCH	针对不与用户交互的批处理任务
	CFS	SCHED_IDLE	针对优先级最低的后台任务

Priority in Linux Kernel

■ 两级优先级调度机制—两级队列调度机制

- ▣ 1. 任务根据自身性质，分属不同调度类，各调度类优先级
 - 面向硬实时的Deadline调度类DL，面向软实时的RT调度类，非实时调度类CFS
 - 调度类优先级及调度顺序：DL > RT > CFS, 优先调度实时DL类任务，再依次调度实时RL、非实时CFS类任务，DL → RT → CFS

(DL, EDF)	SCHED_DEADLINE	>
(RT, FIFO/RR)	SCHED_FIFO/SCHEDRR	>
(CFS)	SCHED_NORMAI/SCHED_OTHER/SCHED_BATCH	>
	SCHED_IDLE	

- ▣ 2. 同一调度类内，按照任务优先级和调度算法，进行调度

Priority in Linux Kernel

- 两类任务优先级
 - ▣ 内核空间中，用于进程调度的四种任务优先级
 - 动态优先级prio，静态优先级static_prio，实时优先级rt_prio
 - 归一化优先级normal_prio
 - ▣ 用户空间中可观察的用户优先级
 - NI, PRI。NI: Nice值，修改、设置进程优先级的参数
 - 使用htop、top命令可以观察
- 进程task_struct中与四种（内核）任务优先级相关的4个字段

```
struct task_struct {  
    ... ..  
    int          prio, static_prio, normal_prio;  
    unsigned int  rt_priority;  
    ... ..  
}
```

Priority in Linux Kernel

	CFS	RT	DL
nice	[-20,19]		
prio/normal_prio	=static_prio[100,139]	=max_rt_prio - 1 - rt_priority[0,99]	-1
static_prio	=max_rt_prio + nice +20[100,139]		
rt_priority	0	[0,99]	0

调度算法及优先级使用方式：

对非实时任务，CFS调度算法使用static_prio，为任务分配在1个调度周期内所占时间片

对DL实时任务，EDF调度算法使用任务的deadline进行调度，不使用四类优先级

面向普通非实时类进程的static_prio单调减，即值越大，优先级越低；面向实时类进程的rt_priority单调增，值越大，优先级越大。将这两类优先级转为normal_priority，便于比较不同类任务的优先级

Priority in Linux Kernel

■ 动态优先级prio

- ▣ 可在任务运行过程中动态改变，通过内核函数`effective_prio`计算得到
- ▣ 当低优先级任务长时间得不到CPU时，调度器将暂时调高其动态优先级，防止饿死
- ▣ 在 CFS 类或者 DL/EDF 类的调度策略中，与 `normal_prio` 是一致的
- ▣ 在 RT 策略中，直接返回当前的动态优先级值，值与静态优先级相同

Priority in Linux Kernel

■ 静态优先级static_prio

- ▣ 面向采用CFS 调度器的非实时类任务
- ▣ 范围100~139，值越小，优先级越高，权重weight越大
- ▣ CFS调度算法根据各个任务的静态优先级和权重weight，决定分配给任务的时间片长度
- ▣ 计算：根据任务的nice值，通过宏定义公式

$$\text{nice} + \text{MAX_USER_RT_PRIO} + (\text{MAX_NICE} - \text{MIN_NICE} + 1) / 2$$

$$= \text{nice} + 120$$

```
#define MAX_NICE      19
#define MIN_NICE      -20
#define NICE_WIDTH    (MAX_NICE - MIN_NICE + 1)
```

■ Nice

- ▣ 用户空间任务优先级
- ▣ 范围: -20~19，值越大优先级越低
- ▣ 可由用户设定

```
#define MAX_USER_RT_PRIO    100
#define MAX_RT_PRIO        MAX_USER_RT_PRIO

#define MAX_PRIO            (MAX_RT_PRIO + NICE_WIDTH)
#define DEFAULT_PRIO        (MAX_RT_PRIO + NICE_WIDTH / 2)
#define NICE_TO_PRIO(nice)  ((nice) + DEFAULT_PRIO)
#define PRIO_TO_NICE(prio)  ((prio) - DEFAULT_PRIO)
```

- 实时优先级rt_priority
 - ▣ 面向RT 调度算法类任务的优先级
 - ▣ 值越大，优先级越高，取值范围 1~99
 - ▣ 用户可以使用sched_setparam 等函数进行设置

Priority in Linux Kernel

- 归一化优先级normal_prio
 - ▣ 将不同的调度类对应的实时、非实时任务的各种优先级（静态、动态、实时）归一化到统一尺度，便于相互间比较
 - ▣ 任务的归一化优先级值越小，代表实际优先级越高，最小值是 -1

任务调度类	Deadline(EDF)	RT(FIFO/RR)	CFS(Other/Normal/Batch)
任务normal_prio	-1	0~98	100~139
计算公式	$\text{MAX_DL_PRIO} - 1 = -1$	$\text{MAX_RT_PRIO} - 1 - \text{rt_priority}$	$\begin{aligned} \text{static_prio} &= \text{NICE} + \text{MAX_USER_RT_PRIO} \\ &+ (\text{MAX_NICE} - \text{MIN_NICE} + 1) / 2 \\ &= \text{NICE} + 120 \end{aligned}$
参数设置/范围	宏定义 MAX_DL_PRIO 0	1. 宏定义 MAX_RT_PRIO 100, 2. $\text{rt_priority} \in [1, 99]$ ，值越大，实时优先级越高	1. 宏定义 MAX_NICE 19, MIN_NICE -20 2. $\text{NICE} \in [-20, 19]$ ，值越小，优先级越高 3. 宏定义 MAX_USER_RT_PRIO 100

Priority in Linux Kernel

- 用户空间优先级NI, PRI
 - ▣ 使用 htop、top 命令，可以观察NI、PRI的值
 - ▣ 取值范围 -20~19
- NI, nice 值
 - ▣ 表示进程的友好程度。NI 值越大，任务越友好、越慷慨，会使用相对较少的 cpu 时间，即优先级低；NI值低的任务优先级高，使用较多的 cpu 时间
- PRI, 用户空间观察到的任务优先级
 - ▣ 不对应内核空间中前四种优先级
 - ▣ 值跟随 NI 值变化
 - ▣ 在 CFS 调度时， $PRI = 20 + NI$ ， $PRI(new) = PRI(old) + nice$
 - ▣ 用户空间进程刚初始化时，默认 $PRI=20$ ， $NI=0$

Deadline调度类

- 使用函数sched_setattr，将一个进程的调度类/调度策略设置/切换为DEADLINE

```
struct sched_attr attr;
```

```
memset(&attr, 0, sizeof(attr));
```

```
attr.size = sizeof(attr);
```

```
/* This creates a 200ms / 1s reservation */
```

```
attr.sched_policy = SCHED_DEADLINE;
```

```
attr.sched_runtime = 20*1000*1000;
```

```
attr.sched_deadline = attr.sched_period = 33*1000*1000;
```

```
ret = sched_setattr(0, &attr, flags);
```

各类任务的归一化优先级normal_prio计算方法

```
static inline int normal_prio(struct task_struct *p)
{
    int prio;
    if (task_has_dl_policy(p))
        prio = MAX_DL_PRIO-1;
    else if (task_has_rt_policy(p))
        prio = MAX_RT_PRIO- 1 - p->rt_priority;
    else prio = _normal_prio(p);
    return prio;
}
```

Windows Scheduling

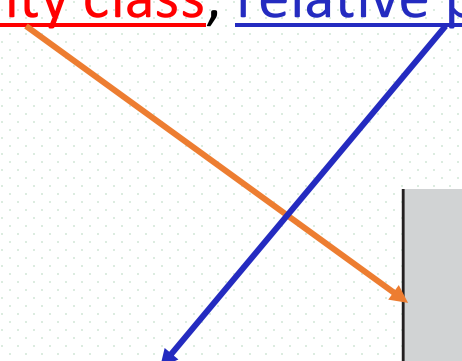
- Windows uses **priority-based preemptive** scheduling
- Highest-priority thread runs next
- **Dispatcher** is scheduler
- Thread runs until
 - (1) blocks /被阻塞，如进入I/O操作，放弃CPU
 - (2) uses time slice /时间片运行完
 - (3) preempted by higher-priority thread /被高优先级线程抢占
- Real-time threads can preempt non-real-time
- 32-level priority scheme
 - **Variable class** is 1-15, **real-time class** is 16-31
 - Priority 0 is memory-management thread
- Queue for each priority
 - 每个优先级设置一个调度队列，存放具有该优先级的、需要调度的threads
- If no run-able thread, runs **idle thread**

Windows Priority Classes

- Win32 API identifies several **priority classes** to which a process can belong
 - ▣ REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS, NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS
 - ▣ All are variable except REALTIME
- A thread within a given priority class has a **relative priority**
 - ▣ TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
- Priority class and relative priority combine to **give numeric priority**
 - ▣ thread根据其priority class和relative priority, 计算其数值优先级, 用于调度
- Base priority is NORMAL within the class
- If quantum expires, priority lowered, but never below base
 - ▣ 当运行在CPU上的thread的时间片运行完后, thread的优先级将被降低
 - 该thread后续再次被调度的几率降低, 该thread将CPU让给其它thread

Windows Priorities

- priority class, relative priority, 数值优先级 **numeric priority-值越大, 优先级越高**



	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

Windows Priority Classes (Cont.)

- If wait occurs, priority boosted depending on what was waited for
 - ▣ 当thread进入waiting态发生等待时，OS会提升其优先级，便于当该thread重新进入ready态后，尽快被调度
- Foreground window given 3x priority boost
 - ▣ 对交互性强的前台thread，其优先级被提升3倍
- Windows 7 added **user-mode scheduling (UMS)**
 - ▣ Applications create and manage threads independent of kernel
 - ▣ For large number of threads, much more efficient
 - ▣ UMS schedulers come from programming language libraries like C++ **Concurrent Runtime** (ConcRT) framework

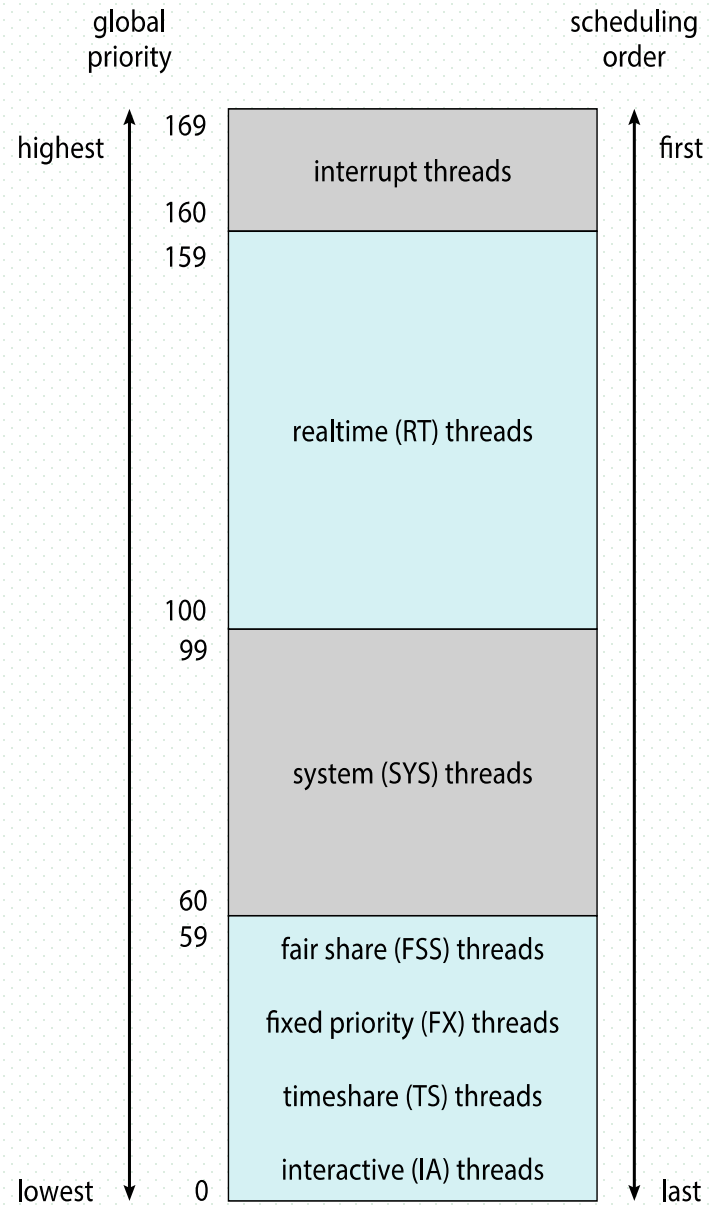
- Priority-based scheduling
- Six classes available
 - ▣ Time sharing (default) (TS)
 - ▣ Interactive (IA)
 - ▣ Real time (RT)
 - ▣ System (SYS)
 - ▣ Fair Share (FSS)
 - ▣ Fixed priority (FP)
- Given thread can be in one class at a time
- Each class has its own scheduling algorithm
- Time sharing is multi-level feedback queue
 - ▣ Loadable table configurable by sysadmin

Solaris Dispatch Table

数值越大 优先级越低 priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59



Solaris Scheduling



Solaris Scheduling (Cont.)

- Scheduler converts class-specific priorities into a per-thread global priority
 - ▣ thread with highest priority is selected to runs next
 - ▣ thread runs until
 - (1) be blocked,
 - (2) uses time slice, and the slice expires
 - (3) preempted by higher-priority thread
 - ▣ multiple threads at same priority are selected via RR

5.8 Algorithm Evaluation

- How to select CPU-scheduling algorithm for an OS?
- Determine criteria, then evaluate algorithms
- Four strategies for evaluating scheduling algorithms
 - ▣ deterministic modeling and analytical evaluation
 - taking a particular predetermined **workload/工作负荷、工作量** and defines the performance of each algorithm for that workload
 - ▣ queuing models
 - ▣ implementing algorithms and applying them to practical systems, observe and evaluate their performance
 - ▣ evaluation of CPU Schedulers by simulation

Appendix 5-1 基于指数平均的CPU burst预测

Exponential Averaging prediction of next CPU bursts

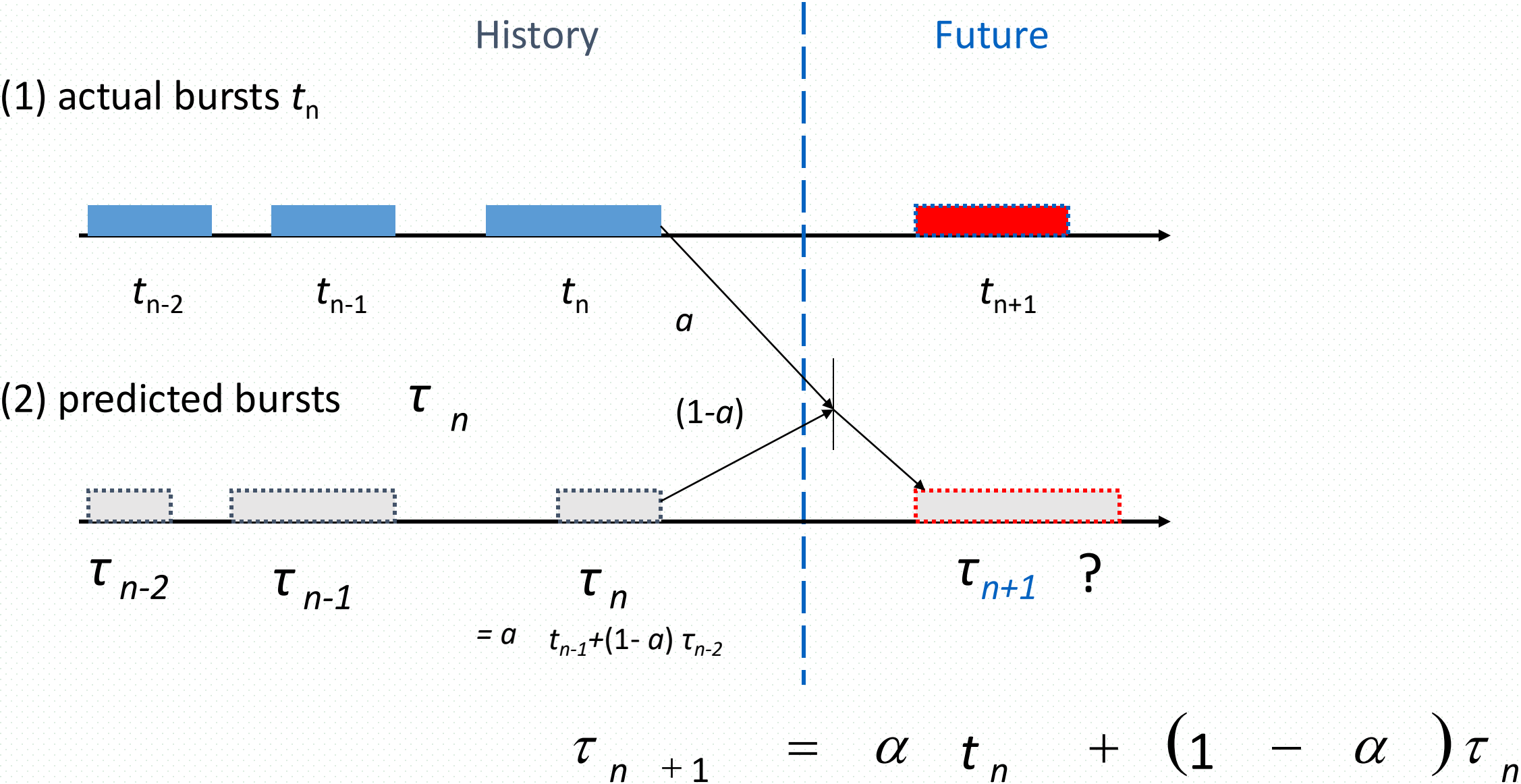
- Define

1. t_n = actual length of n^{th} CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. $\alpha, 0 \leq \alpha \leq 1$

- Then

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$$

Exponential Averaging prediction of next CPU bursts



Exponential Averaging prediction of next CPU bursts

- Examples of Exponential Averaging

- ▣ $\alpha = 0$

$\tau_{n+1} = \tau_n$, recent history does not count

- ▣ $\alpha = 1$

$\tau_{n+1} = t_n$, only the actual last CPU burst counts

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$$

- If we expand the formula, we get

- ▣
$$\begin{aligned} \tau_{n+1} = & \alpha t_n + (1 - \alpha) \alpha t_{n-1} + \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ & + (1 - \alpha)^{n+1} \tau_0 \end{aligned}$$

- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor

Appendix 5-2 Priority-based scheduling in Unix

- 基于优先级/权/数的调度
 - 广泛采用的一种进程调度算法
 - 由系统（或用户、或系统与用户结合）赋予每个进程一个优先级，在CPU空闲或调度程序启动时，进程调度程序从就绪进程中选择一个优先级最大（或者最小）的进程占用CPU（该进程就从就绪状态转换成运行状态）
- 关键
 - 如何确定进程的优先级
 - 进程的优先级确定之后是固定的，还是随着该进程运行的情况的变化而变化

Priority-based scheduling in Unix

■ 两种方法确定优先级

▣ 1. 静态优先级

- 进程的优先级在进程创建时确定后就不再变化
- 进程优先级确定

(1) 系统确定：运行时间、使用资源，进程的类型

(2) 用户确定：紧迫程度，计费与进程优先级有关

(3) 系统与用户结合：

用户可以为本用户进程设置优先级，但不作为调度的唯一依据，系统将用户设置的进程优先级作为确定进程优先级的一个参数

▣ 2. 动态优先级

- 系统在运行的过程中，根据系统的设计目标，不断地调整进程的优先级
- 优点：客观地反映进程的实际情况，保证达到系统设计目标

Priority-based scheduling in Unix

■ UNIX系统的进程调度

- ❑ UNIX系统采用优先级调度算法，每个进程有一个进程优先值`p_pri`，`p_pri`是`proc`结构中的一个变量，其取值范围是 $-127 \sim 127$ ，其值越小，进程优先级越高
- ❑ 调度程序从就绪状态的进程中，选择一个优先值最小、优先级最高的进程占用CPU

■ 进程优先数计算公式

$$p_pri = \min \{127, (p_cpu/16 + p_nice + PUSER) \}$$

- ❑ `p_cpu` 进程占用CPU的程度，到目前为止已经在CPU上运行的时间
- ❑ `p_nice` 用户通过系统调用`nice (priority)` 设置的进程优先数
- ❑ `PUSER` 常数，其值为100

Priority-based scheduling in Unix

- 进程进入高优先级睡眠的原因
 - ▣ 0 # 进程 (- 100优先数)
 - ▣ 因资源请求得不到满足的进程, 磁盘 (- 80) , 打印机 (- 20) , ...;
 - ▣ 等待块设备I/O完成的进程, (- 50)
- 进程进入低优先级睡眠的原因
 - ▣ 因等待字符设备I/O完成的进程, (0 ~ 20的优先数)
- 处于用户态运行进程, 其优先数一般情况下大于100

Appendix 5-3 Scheduling in Linux

- O(1) scheduling in version 2.5
 - ▣ multi-level queues
- Scheduling strategies and CFS in Version 2.6.23 + 
 - ▣ CFS, Completely Fair Scheduler

O(1) Scheduling in Version 2.5

- 任务类型
 - ▣ 实时任务, real-time
 - ▣ 非实时任务, time-sharing, 如交互式任务
- 两级调度机制
 - ▣ task进入（多CPU）系统, 由全局scheduler根据系统全局情况, 如各个CPU的负载, 将task分配到某个CPU核
 - ▣ 之后, 该任务一直在该CPU核上运行, 由核上的本地scheduler管理, 分配该CPU核上的时间片, 不会迁移到其它CPU核
 - CPU核亲和性
- 基于优先级的、可抢占式调度
 - ▣ 实时Real-time任务, 优先级 range from 0 to 99, 任务nice value from 100 to 139
 - 根据nice值, 确定优先级
- 实时、非实时任务获得CPU时间片, 运行在CPU上。任务的优先级越高, 获得的时间片越长

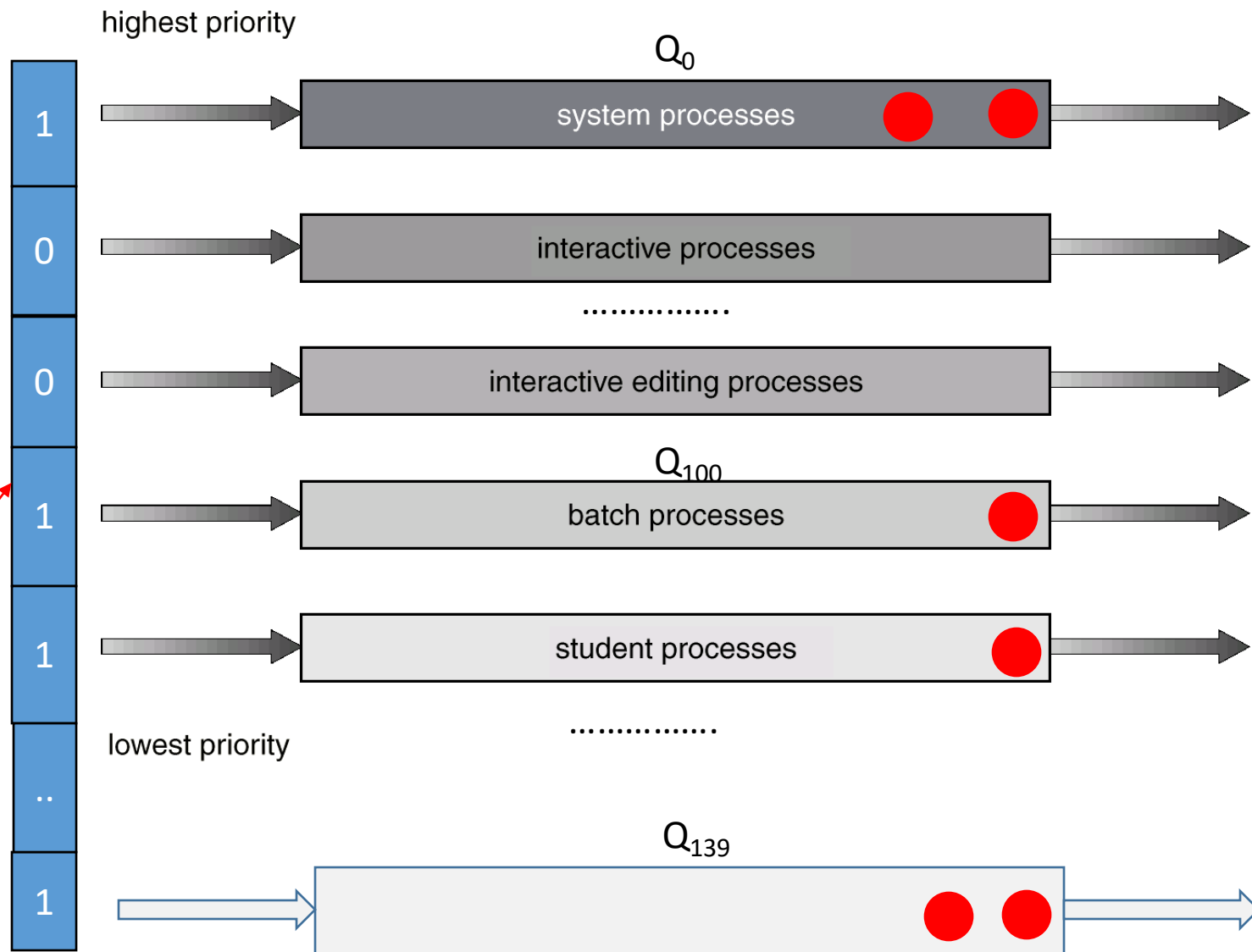
O(1) Scheduling

- 每个CPU核维护一个各自独有的**本地运行队列**，队列中的任务仅在该核上调度，运行分配给它的时间片
- 非实时任务的**时间片**根据任务的**Niceness**值（优先数）确定
- **本地运行队列**由**两个多级任务队列**组成
 - 激活队列(active queue)
 - 还没有分配时间片、等待调度的任务
 - 已经分配时间片开始运行、单当前时间片没有运行完、仍有时间片剩余的task，e.g. 在执行该时间片的过程中，由于发生被高优先级task抢占等情况，处于“停顿”状态
 - 过期队列(expired queue)
 - 本次分配给task的时间片已耗尽的task，但该task还需要执行后续bursts

O(1) Scheduling

- 一个运行在CPU上的task的时间片耗尽后，该任务由激活队列加入过期队列
- 当激活队列中没有可调度的task，**两个队列的角色互换**，开始新一轮调度
- 据此，进入系统的各个任务依次获得多个时间片，完成其各自的多个CPU bursts

- 140级优先级, 0~139
- 任务分为real-time任务、非实时的time-sharing task
 - ▣ 高优先级[0, 99], 对应实时任务
 - ▣ 低优先级[100,139], 对应非实时任务
- 激活、过期这两个多级队列都各有140个优先级, 每个优先级队列对应一种优先级级的任务
- 每个多级队列维护一个**位图 bitmap**, 图中比特位记录了对应的队列中是否有任务等待调度(ready, running)

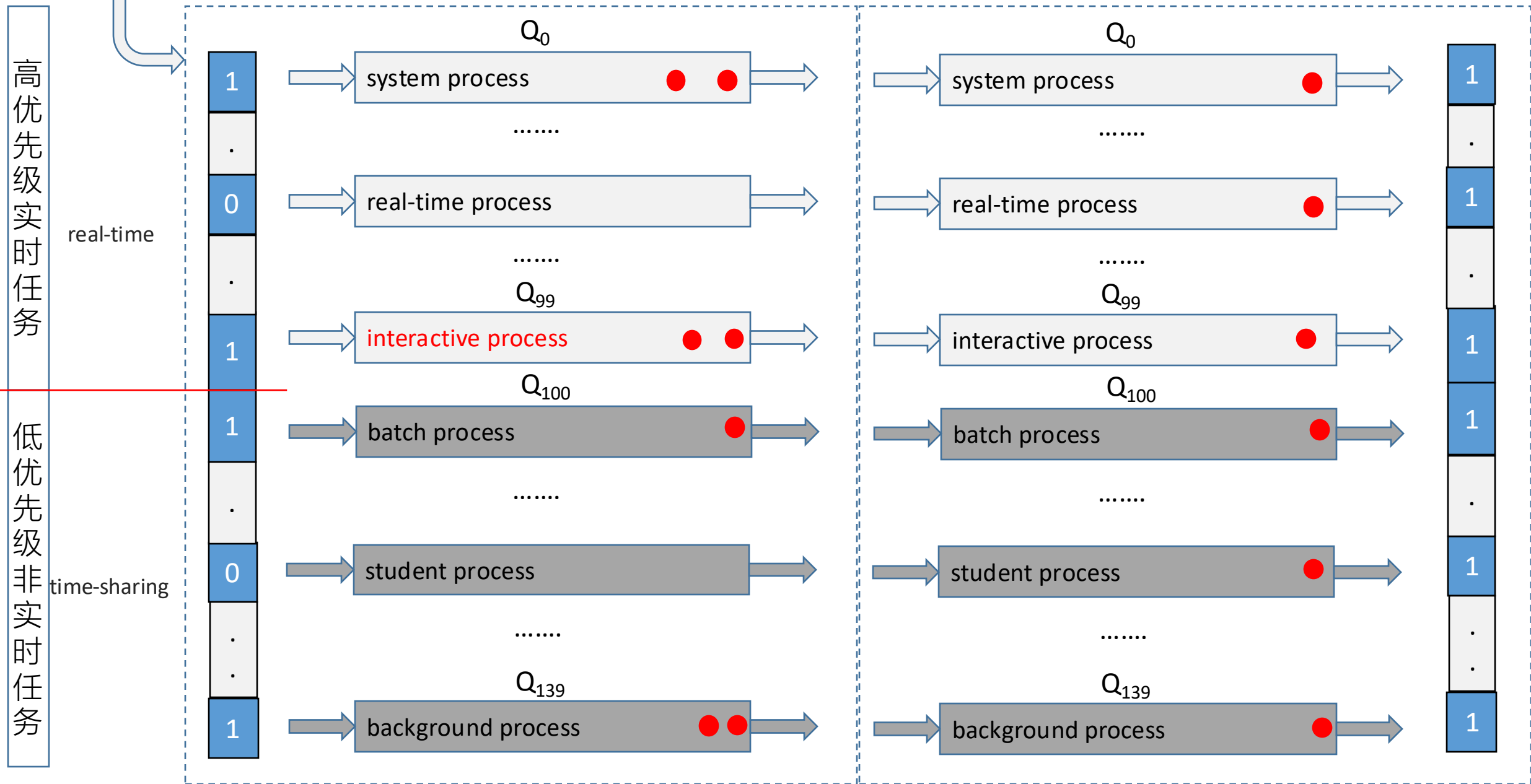


13 tasks

active queues (7 tasks)

队列互换

expired queues (6 tasks)



O(1) Scheduling

- O(1)调度器根据位图找到激活队列中优先级最高的第一个队列【花费常数时间】，并调度该队列中第1个任务【花费常数时间】，如 Q_0 中的第1个任务，其时间复杂度为O(1)，与运行队列中任务数目无关
- 关于交互式任务处理
 - 为防止交互式任务在时间片用完后，需要等待所有其它任务的时间片用完后，才能再次调度执行，导致相应时间较长，交互式任务在时间片用完后，仍然加入激活队列
 - 为防止交互式任务过于激进，长期运行，导致过期队列中的任务无法执行，当过期队列中任务等待了过长时间后，调度器将交互式任务加入过期队列

O(1) Scheduling

- 特点
 - ▣ 支持SMP, 处理器亲和, 负载均衡
- 缺点
 - ▣ 1. 交互式任务判定过于复杂, 使用大量启发式, 交互式任务的响应时间无法完全保障
 - poor response times for interactive processes
 - ▣ 2. 静态时间片
 - 非实时任务的时间片根据任务的Niceness值（优先数）确定
 - e.g. , Niceness=-20, 0, 19 的非实时任务, 对应的时间片分别为800ms、100ms、5ms
 - 随着系统中任务数增加, 任务的调度时延增大

CFS原理

- 为系统内各个task分配CPU时间时应保证平衡/公平性，当分给某个task的CPU时间失去平衡时，即该任务相对于其他任务而言未被分配足够的CPU时间，应给失去平衡的任务分配CPU时间，让其执行
- 使用虚拟运行时间vruntime来维持、评估提供给task的CPU时间量
 - ▣ 任务的vruntime越小，其对CPU的需求越高，应优先分配CPU
- 不直接使用优先级为任务分配CPU，而是将其用作允许task执行的时间的衰减系数
 - ▣ 低优先级task具有更高的衰减系数，而高优先级task具有较低的衰减系数
 - ▣ 与高优先级任务相比，低优先级任务允许任务执行的时间消耗得更快
 - ▣ 避免维护按优先级调度的运行队列
- 被调度的任务存储在以时间为顺序的红黑树中，对CPU需求最多、vruntime最小的任务存储在树的左侧，CPU需求最少、具有最大vruntime的任务存储在树的右。调度器选择位于最左端的任务，分配CPU。

Scheduling Strategies and CFS in 2.6.23 +

- Linux任务task根据其特性，分为非实时（批处理）、交互式、实时任务等，不同类型任务采用不同调度策略和调度器
- 用户可以设置任务采取的调度策略和对应的调度器
- 3个非内核task使用的调度器DL、RT、CFS，以及对应的调度任务、调度策略如下

调度器	调度策略	描述及适用任务
DL	SCHED_DEADLINE	针对实时任务，类似EDF的调度策略
RT	SCHED_FIFO	针对实时任务，执行直至结束，或被更高优先级任务抢占
RT	SCHED_RR	针对实时任务，时间片轮转
CFS	SCHED_OTHER	公平的分时调度策略
CFS	SCHED_BATCH	针对不与用户交互的批处理任务
CFS	SCHED_IDLE	针对优先级最低的后台任务

- 在上述三种调度器中，Linux优先调度DL调度器管理的任务，其次是RT调度器管理的任务，最后才是CFS调度器管理的任务
- 顺序：DL → RT → CFS

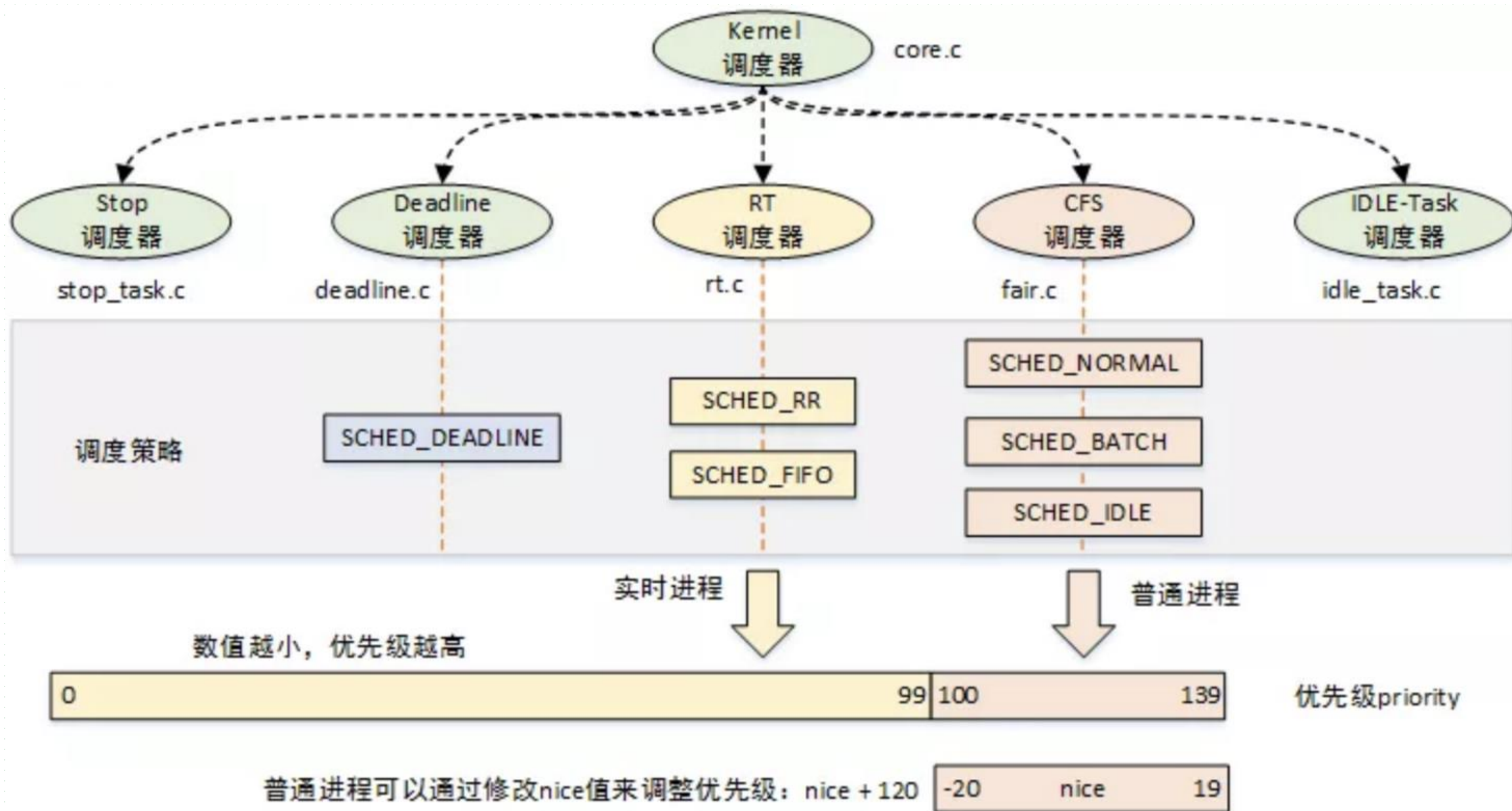
Scheduling Strategies and CFS in 2.6.23 +

- Linux内核提供5个调度器，使用 `struct sched_class`对调度器进行抽象，调度器代码由6个调度类组成
 - ▣ **Stop调度器**， `stop_sched_class`
 - 优先级最高的调度类，可以抢占其他所有进程，不能被其他进程抢占
 - ▣ **Deadline调度器DL**， `dl_sched_class`
 - 使用红黑树，将任务按照绝对截止时间**deadline**进行排序，优先选择调度最小**deadline**的任务
 - ▣ **RT调度器**， `rt_sched_class`
 - 实时调度器，为每个优先级维护一个队列，采用RR、FIFO调度策略
 - ▣ **CFS调度器**， `fair_sched_class`
 - 完全公平调度器，采用完全公平调度算法，引入虚拟运行时间概念
 - ▣ **IDLE-Task调度器**， `idle_sched_class`
 - 空闲调度器，每个CPU都有一个**idle**线程，当没有其他进程可以调度时，调度运行**idle**线程

Scheduling Strategies and CFS in 2.6.23 +

- **Stop调度器和IDLE-Task调度器**，仅由内核任务使用
- Stop调度器
 - 所属调度器类为stop_sched_class，使用Stop调度器的内核进程具有最高优先级
 - 用于停止CPU，一般用于SMP系统，实现负载平衡和CPU热插拔

五种Linux调度器



Linux调度策略

- OS内核提供以下调度策略，用户程序/task可调度策略和选择调度器。但Stop调度器和IDLE-Task调度器，仅由内核task使用
 - SCHED_DEADLINE
 - 截止期deadline调度策略，使task选择Deadline调度器来调度运行
 - SCHED_RR
 - 实时进程的调度策略，采用时间片轮转调度方法。进程用完时间片后，加入进程优先级对应的运行队列的尾部，将CPU让给队列中同优先级的其它进程
 - SCHED_FIFO
 - 实时进程调度策略，采用FIFO先进先出调度方法
 - SCHED_NORMAL/Other
 - 普通进程调度策略，选择CFS调度器
 - SCHED_BATCH
 - 普通进程调度策略，批量处理，选择CFS调度器
 - SCHED_IDLE：普通进程调度策略，使task以最低优先级选择CFS调度器

- 优先级设置
 - RT进程优先数: [0..99];
 - 普通进程优先数: [100..139], 默认值120
 - 位置include\linux\sched\prio.h
- 用户可以采用sched_setscheduler等函数, 选择、设置用户任务采用的调度策略和优先级
- E.g. 对RT任务, 设置的RT优先级数值直接写入任务的task_struct的 task->rt_priority 里面, RT任务的优先级
 - $\text{task->prio} = 99 - \text{task->rt_priority}$
 - , task->prio的数值越小, RT优先级越高

Scheduling Strategies in 2.6.23 +

- 采用DL调度器的实时任务
 - 针对时间要求最紧迫的实时任务
 - 采用类似“最早截止时间优先”EDF的调度策略
- 采用RT调度器的实时任务
 - 以多级优先队列的方式进行调度
 - 对应优先级相同的任务，根据任务的调度策略（_FIFO, _RR），采取相应调度方法
 - FIFO：执行直至结束，或被更高优先级任务抢占
 - RR：时间片轮转
 - 优先级确定：task具有实时优先级/静态优先级sched_priority，数值越高，优先级越大
 - _FIFO, _RR实时任务的sched_priority：范围[1,100)
 - 非实时任务sched_priority：0

Scheduling Strategies in 2.6.23 +

- 采用完全公平调度器CFS的非实时任务
 - Completely Fair Scheduler CFS
 - 引入虚拟运行时间vruntime，使用红黑树管理被调度的tasks
 - 利用task的Niceness值（[-20,19)之间）确定任务占用CPU的份额(shares)
 - Niceness值越小，越不友善，越倾向于占用更多CPU时间

调度时机

- 内核调度器选择处于RUNNING状态的任务进行调度，其它状态下的任务不会被调度
- 内核调度时机如下
 - 调用condresched()时
 - 显式调用schedule()时
 - 从系统调用或者异常中断返回用户空间时
 - 从中断上下文返回用户空间时
- 如果开启内核抢占（默认开启）时，会多出如下几个调度时机
 - 在系统调用或者异常中断上下文中调用preempt_enable()时
 - 多次调用preempt_enable()时，系统只会在最后一次调用时发生调度
 - 在中断上下文中，从中断处理函数返回到可抢占的上下文时
 - 指的是中断下半部，中断上半部实际上会关中断，而新的中断只会被登记，由于上半部处理很快，上半部处理完成后才会执行新的中断信号，这样形成中断可重入

Completely Fair Scheduler CFS原理

- 从Linux2.6.23版本起，CFS调度器进入Linux主线，随着版本从2.6.23演进到5.x，CFS调度器经受住考验，同时也在不断改进完善
- CFS原理
 - ▣ 公平共享(fair-share)调度策略，虚拟运行时间
 - ▣ nice-weight-占用CPU份额share
 - ▣ 调度过程
 - 确定调度周期
 - 确定被调度任务的动态时间片
 - 更新任务虚拟运行时间vruntime
 - 采用红黑树组织被调度任务

Completely Fair Scheduler CFS原理

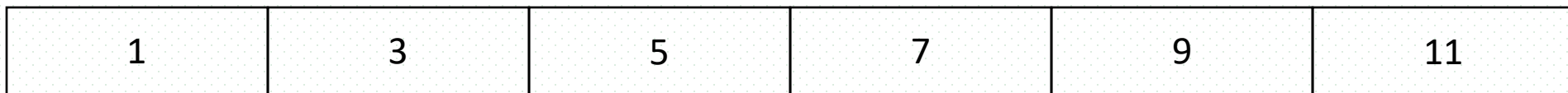
- 1. 采用公平共享(fair-share)调度策略——步幅调度(stride scheduling)，以虚拟运行时间作为任务调度依据
 - 根据任务task的份额(share)，确定任务占用CPU时间之比，份额大则占用CPU时间多
 - e.g. 任务A、B的share之比为6:5，它们占用CPU的比例也是6:5
 - 引入任务**虚拟运行时间**(virtual running time, vruntime)，任务需要记录其已执行的虚拟时间，并在任务被调度时更新其虚拟时间，更新的幅度为步幅stride
 - 调度时，调度器选择具有最小vruntime的任务，为其分配CPU
 - 任务的份额之比对应任务的步幅的倒数之比，任务被调度的次数与其份额成正比

示例：公平共享(fair-share)调度策略——步幅调度(stride scheduling)

- 任务A和任务B，份额share之比为6:5，占用CPU之比也应为6:5
- 调度器从A、B中选择vruntime小的任务，分配CPU
- A、B一旦被选中，运行相同长度的时间片，均为T秒
- A每次被调度，其虚拟时间增加5（步幅为5），B每次被调度，其虚拟时间增加6（步幅为6）
- A、B被调度次数为6:5，占用CPU运行时间之比为6T: 5T

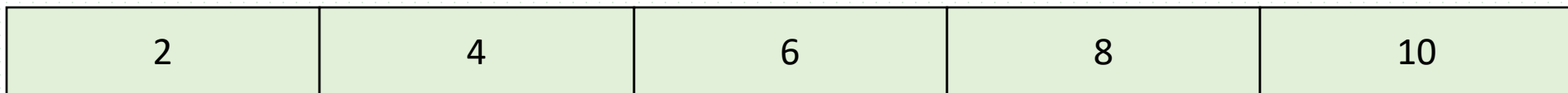
Maxstride: 30

任务A
份额: 6
步幅: 5



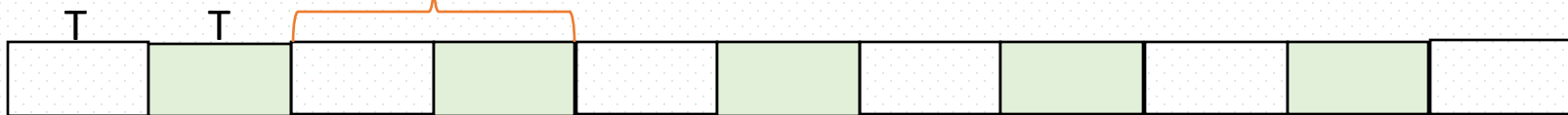
方框内数字：
代表任务被真实执行时的调度顺序

任务B
份额: 5
步幅: 6



虚拟时间

调度周期sched_period=2T



真实运行时间

CFS原理

- 2. 用户可以根据被调度任务的性质，设置其Niceness值，Linux内核静态设置了Niceness值与任务权重（weight，即份额share）的关系
 - ▣ Niceness值越低，权重weight越大，任务占用的CPU份额越大
- 3. 引入调度周期sched_period概念，采用动态时间片，在一个调度周期内，运行队列中每个任务都被调度一次，最坏情况下任务的调度时延即为调度周期
 - ▣ 3.1 确定调度周期sched_period
 - ▣ 3.2 确定被调度任务的动态时间片time_slice
 - ▣ 3.3 更新任务虚拟运行时间vruntime

CFS原理：确定调度周期sched_period

- 调度周期sched_period
 - = 运行队列中全部任务各自执行一个分配给它的时间片所需的总时间
- 各任务的CPU占用之比
 - = 一个调度周期内，各任务运行时间片长度在该周期内占比
- 调度周期相关参数
 - ▣ 调度周期默认值sched_latency, 6ms
 - ▣ 任务平均时间片最小值sched_min_granularity, 0.75ms
 - ▣ 当前被调度任务总数nr_running
- 调度周期设置时的考虑因素
 - ▣ 调度周期设置过长，增大任务的调度时延，影响任务的响应性
 - ▣ 调度周期过短，在被调度任务的数目比较多的情况下，每个任务获得的时间片过短，进程上下文切换频繁，调度管理开销大

CFS原理：确定调度周期sched_period

- 调度周期sched_period设置方法

- ▣ 计算

调度周期默认值 $\text{sched_latency} / \text{nr_running}$

- ▣ 如果 $\text{sched_latency} / \text{nr_running} \geq \text{sched_min_granularity}$ (时间片最小值)
则 sched_period 设置为 sched_latency
否则 sched_period 设置为 $\text{sched_min_granularity} * \text{nr_running}$
//保证时间片至少0.75ms

CFS原理：确定被调度任务的动态时间片time_slice

- 根据确定的调度周期，调度器计算当前运行队列中，第i个任务的动态时间片time_slice_i

$$time_slice_i = sched_period * \frac{weight_i}{weight_rq}$$

- weight_i代表第i个任务的权重，weight_rq代表当前运行队列中全部任务的权重之和

CFS原理：更新任务虚拟运行时间vruntime

- 每个任务的动态时间片可以是不同的，根据任务权重进行了缩放
- 任务每次被调度、执行完分配给它的时间片后，需要对应地更新其虚拟运行时间

$$vruntime_i = vruntime_i + \frac{weight_nice0}{weight_i} * realruntime$$

- $weight_nice0=1024$ ，表示一个Niceness值为0、权重最大的任务的权重。以该权重与本任务权重 $weight_i$ 的比值为系数，将任务实际运行时间 $realruntime$ 映射为虚拟运行时间
- 通过该系数，CFS保证了具有不同动态时间片的任务在执行完自己的时间片后，它们虚拟时间的增幅是一致的

CFS原理：更新任务虚拟运行时间vruntime

- E.g. 任务A的Niceness为0，任务B的Niceness为5
 - 根据nice-weight计算公式，A的权重 $weight_A$ 约为B的权重 $weight_B$ 的3倍
 - 在一次调度周期内，A的动态时间片长度约为B的动态时间片长度的3倍
 - 当A、B都执行完一个时间片后，A、B的vruntime增幅是一样的

CFS原理

- 4. 位于运行队列中的被调度任务组织为红黑树，以vruntime作为索引键值
 - ▣ 以 $O(1)$ 时间快速找到vruntime最小的任务，作为被调度对象
 - ▣ 以 $O(\log N)$ 代价，在红黑树中插入新的任务， N 为红黑树的键值数量，即参与调度的任务数目



Thanks for your
attention



北京邮电大学